FACULTY OF SCIENCE, ENGINEERING, AND COMPUTING

School of *Computing Science and Mathematics*

BSc DEGREE

IN

Software Engineering

PROJECT DISSERTATION

Tyler Watts

K1328289

Implementing a Project Management System

Project Type: Build

Date: 15/04/16

Supervisor: Graeme Jones
Website link: http://project.tylerdev.co.uk

Word count: 16572

Kingston University London

Plagiarism Declaration

Declaration

I have read and understood the University regulations on plagiarism and I understand the meaning of the word *plagiarism*. I declare that this report is entirely my own work. Any other sources are duly acknowledged and referenced according to the requirements of the School of Computer Science and Mathematics. All verbatim citations are indicated by double quotation marks ("…"). Neither in part nor in its entirety have I made use of another student's work and pretended that it is my own. I have not asked anybody to contribute to this project in the form of code, text or drawings. I did not allow and will not allow anyone to copy my work with the intention of presenting it as their own work.

Date 24/04/2017

Signature _____

Plagiarism Declaration 2
1.0 – Introduction 5
1.1 – Background 5
1.2 – Aim and Objectives 5
2.0 - Literature Review of Data Protection & Security 6
2.1 - Introduction to Data Protection 6
2.2 - Principles of Data Protection Act 6
2.3 – Offenses 7
2.4 – Exemptions 9
2.5 –Compliance 9
2.6 – Conclusion 10
3.0 - Requirements Analysis 11
3.1 – Software Development Methodology 11
3.2 - Stakeholder Analysis 13
3.3 - SWOT Analysis 14
3.4 – User Stories 16
3.4 - Use-Cases 17
3.5 – Use-Case Scenarios 19
3.6 - Functional & Non-Functional Requirements 22
3.7 - Discussion 25
4.0 - Design Chapter 26
4.1 - Introduction 26
4.2 - Data Modelling 26
4.2.2 - Revised Entity Relational Diagram 27
4.3 - UX & Interfaces 35
Conclusion 42
5.0 - Implementation Chapter 43
5.1 - Tools & Technologies 43
5.2 – Architecture 44
5.3 – Building the Graphical User Interface (GUI) 45
5.4 – Implementing the Database 45
5.5 – Building System Functionality 47
5.6 – HTTPS & SSL 58
5.7 – Summary 59
6.0 - Testing Chapter 60
6.1 – Unit Testing 60
6.2 – Functional Testing 60
6.3 – Non-Functional Testing 60
6.4 – Test Cases 60
6.5 – Usability Testing 66
7.0 - Critical Evaluation 67
7.1 – Challenges Faced 67
7.2 – Project Insights 67
7.3 – Critical Technical Review 67
7.4 – Future Functionality 68
7.5 – Conclusion 68
References & Bibliography 69
Appendices 71
Wireframes 71
Activity Diagrams 72
Use-Case Scenarios 74

Database Tables 77
Project Proposal 78

1.0 – Introduction

1.1 – Background

During the course of this project, I intend to design and develop an employee management web application for the client, Staxoweb Ltd. Staxoweb are a web development and telecoms provider company based in Tolworth, Surrey. They specialise in developing bespoke websites and applications as well as selling infrastructure based products. Having previously been employed by them, they decided it'd be a great opportunity for a developer who fully understands the business, and their current tracking procedures, to create an improved web application to better process these figures.

The need for this software was proposed to them as a possible improvement over their current project and attendance tracking methods. The application will be built on a solid foundation of extensive research, meetings with the client, requirements analysis, design investigation, implementation, and rigorous testing. The software development methodology employed throughout this project will be an adapted version of the iterative-oriented Agile methodologies.

1.2 – Aim and Objectives

Their current system named, '***Timesheets'***, is an employee-based attendance tracking system in which holiday requests can be created. They also currently use a project based tracking system called '***WorkOrders'***. Both of these systems produce basic tracking functionalities, with little to no visual aid on the progress of the business. Their project management system also relies on the honesty of employees who enter the amount of time spent on each project manually. Having consulted with the CTO and CEO, they explained that the figures they're receiving are not consistent and require a more efficient management strategy. This project will further improve and combine the features from both their current systems, into one central work 'hub'. The method in which it will be implemented will further allow for the addition of functionality modules in the future.

The primary aim for this project is to provide a clear and concise application, accessible by each employee type, to improve employee and project management across the entire company. Staxoweb currently feel that they're not utilising these procedures and could potentially be financially impacted by the absence of it.

2.0 - Literature Review of Data Protection & Security

This segment of the report is a review of the rules and regulations contained within the Data Protection Act, regarding individuals' sensitive information and how it's protected online.

The Data Protection Act 1998 is a regulation within the United Kingdom that was sanctioned to administer the protection and security of delicate data. Protection of user-related data has become a mandatory aspect amongst modern web applications, with most systems' being specifically designed to adhere to the principles stated in the Data Protection Act. Failure to adhere to these principles can result in significant fines.

In relation to this project, the data being stored will be processed to allow users access into the system, determine privileges, and display employee-specific information to their relevant managers. Data protection has become necessary as most databases are stored on a webserver. Communication between the client-side and server can be interrupted/interfered by unauthorized third-parties. This review intends to explore and identify the law regarding data protection, relevant protection breaches, exemptions within the data protection act, and how web developers incorporate these laws into their projects.

2.1 - Introduction to Data Protection

The method of governing the enforcement of data protection in the United Kingdom is detailed within a Parliament approved law known as the Data Protection Act 1998. It is an updated edition from the previously applied law, the Data Protection Act 1984.

The most notable alteration from the previously applied law, is the now-implemented inclusion of non-digital based formats. This includes manual paper filing documentation, and any other platform that is classed as non-computerised.

Despite being titled the Data Protection Act 1998, it was not fully implemented until March of the year 2000. It was enforced as a guideline for all organizations and individuals to follow in regards to the obtaining and storing of sensitive data. An individual's default personal information is stated to consist of, but not limited to the following; Full name, date of birth, address of residence, and telephone number. However, the default personal information is not the only data considered to be sensitive, especially within different industrial sectors. Data detailing an individual's ethnicity, religion, physical/mental condition is also considered sensitive.

2.2 - Principles of Data Protection Act

The Data Protection Act consists of 8 core principles. The key purpose of these principles is to ensure protection for individual's sensitive data.

2.2.1 – Data processed fairly & lawfully

This principle is to ensure the intentions of the data controller are legitimate and lawful. An Institution must have sound justifications for the collection and processing of an individual's personal data. It also states that the data must not be handled in a way that invokes adverse effects on said party.

2.2.2 – Personal Data shall be obtained only for lawful purposes.

This principle is to ensure organizations are clear concerning their motives for collecting respective sensitive data, and that the processing of said information is consistent with the expectations of the persons involved. Organizations must comply with the DPA's fair processing requirements and the

user must be displayed with privacy notices and be informed about how and why their data is being stored.

2.2.3 – Personal Data shall be adequate and relevant

This principle is ensuring that the data obtained is not excessive in relation to the purpose for which it is processed. It also ensures that the data being processed is adequate/fit for its purpose.

2.2.4 – Data will be accurate and kept up to date

This principle is applied to make sure procedures are in place to ensure the data collected is as accurate as possible. Organizations should also consider whether specific segments of data require updating and detect suspicious changes with any existing information. ***(Legislation.gov.uk, 2017)***

2.2.5 – Processed Data shall be kept no longer than necessary

This principle is applied to ensure that personal data is not held on to any longer than is necessary, therefore reducing the risk of sensitive information being leaked. It forces organizations to review the length of time personal data is kept, consider the purpose and whether the data is now redundant, and to securely delete information that is presently surplus to requirements.

2.2.6 – Processed in Accordance with the rights of the data protection act.

This principle is applied to ensure that individuals have rights to how their data is processed. It states that individuals have the right to prevent processing for digital marketing, automated processing, object to processing that may cause stress, and claim compensation should their details be leaked and breach the DPA.

2.2.7 – Organization security to prevent data loss, damage etc.

This principle is put in place to ensure the storing and gathering of data is secured and protected through means of strict procedures, appropriate technologies, and well trained staff. It also states that the organization must have a procedure ready to respond in the case of a data breach.

2.2.8 – Sending personal data outside of the EEA (European Economic Area)

This principle states that personal data should not be sent to any country/territory outside of the EEA unless said territory ensures a significant quality of protection for the rights involved with the processing of personal data. - ***(Legislation.gov.uk, 2017)***

2.3 – Offenses

The Data Protection Act (DPA) regulates the handling of information regarding one's personal data. The DPA enforces various offences, of which can only be introduced by the Director of Public Prosecutions (DPP). In regards to the breach of the DPA, there are no custodial sentences. All breaches that result in an offense are penalized in the form of a fine. The DPA also states that search warrants are available to the Information Commissioner, should suspicion need verifying in specific cases***. - (Dataprotection.ie, 2017)***

2.3.1 – Offense Types

There are various ways to breach the DPA, and therefore different offense types. Relating specifically to Section 55 part 1, the unlawful obtaining and processing of personal data, it states that it is a criminal offense and breach of the DPA to recklessly obtain and disclose any sensitive information with the absence of consent from the data controller.

Additionally, in section 55 part 4 and 5, the DPA enforce offences for the selling or intention to sell personal data. It also states that an advertisement for personal data registers as an intention to sell and will be punished as if it was so.

2.3.2 – Offenses within the Industry

As the technological age advances, the need for secure data protection is more relied upon. Looking over past breaches of the Data Protection Act within the Industry, can help clarify how vital data protection is, and emphasize the consequences of breaking this law.

London NHS trust fined for HIV Newsletter

In May 2016, A London NHS trust was charged with the data leak of HIV positive patients. The error occurred when a newsletter intended for these patients was emailed to other patients. Additionally, the addresses were entered into the 'To' field rather than the 'BCC' field, resulting in the email addresses of these patients being shared with everyone that received the newsletter. Over 700 patient's confidential data was leaked and broadcast through the email.

Following an investigation from the ICO, the Chelsea and Westminster Hospital NHS Foundation Trust, which is responsible for the operating of the clinic, was charged with the substantial breach of the Data Protection Act and subsequently fined £180,000.

Further to this investigation, it was discovered that a similar incident occurred in March of 2010, when an email containing a HIV treatment questionnaire was sent to patients with the addresses contained within the 'To' field, resulting in a similar issue on a slightly smaller scale. Information Commissioner Christopher Graham stated "*It is crucial that the senior managers at NHS Trusts understand the requirements of data protection law, and the serious consequences that follow when that law is broken.*" - ***(London, Ico.org.uk, 2016)***

London Ealing council data breach as court bundle lost

In November of 2016, Ealing council was advised to improve its methods of protecting sensitive data, when an employee breached the Data Protection Act by leaving court documents on the roof of their car and proceeding to drive on. Data relating to 27 people, including 14 children was misplaced during the incident and was never recovered.

The ICO (Information Commissioner's Office) carried out an investigation because of the incident, and found that 27% of social workers in the council's children's services department were employed on a temporary basis. It was also discovered that no records were stored in relation to training temporary employees on data protection.

Regarding this case, the punishment did not result in a hefty fine, but instead in the form of compulsory guidelines. This case emphasizes the importance of data protection across multiple platforms. Their electronic data storage system contained all the appropriate security procedures to adhere to the DPA, but was ultimately let down by manual complacency. - ***(Council, Ico.org.uk, 2017)***

Sony PSN data breach

In April 2011, Sony was the target of several electronic attacks from malicious hackers. Both the Playstation Network (PSN) and Qiocity services were affected. Consequently, the PSN got taken offline for a week without a statement from Sony to the consumers. The infiltrators were known to have gained access to the personal and financial data of 101 million accounts.

In addition to the PSN attack of April 2011, Sony were subsequently hit with another breach in October of the same year. This infringement involved the security of 93,000 user's username and passwords. ***- (Bonner, 2012)***

2.4 – Exemptions

As is the case with many legislations, the Data Protection Act 1998 has several exemptions from an individual's right to their data being covered by the core principles. These are referred to as 'special circumstances'. The following circumstances are exempt from the first principle of the data protection act;

- Personal data processed for the prevention or detection of crime [9]
- The apprehension or prosecution of offenders

This is due to the violation of lawfully processed data. Other special circumstances can consist of 'National Security, Health, Education, Social work, and Manual data held by public authorities'. It is important to consider these exemptions when processing data, as it could result in an accidental breach of the Data Protection Act. - ***(Ico.org.uk, 2017)***

2.5 –Compliance

The ICO (Information Commissioner's Office) is a regulatory body with the power to influence the behaviours of an organization that collects and processes personal information. The following powers

2.5.1 – Technological Security Standards

Standardized methods of securing personal data are widely available and have become an industry criteria for organizations to employ. Use of these technologies will help ensure personal data is secure, and will show all efforts are being employed to comply with the Data Protection Act. These technologies apply solely to digital platform of storing/collecting data.

HTTPS – Hypertext Transfer Protocol Secure

HTTP is the protocol of which data is transferred between the browser and the web application/site. The 'S' on the end of HTTPS stands for 'Secure'. HTTP transfers data using plain text, which can be detrimental if an unwanted third party intercepts the connection. HTTPS encrypts the data being transferred, meaning if a third party were to gain access, the data would still be secured. - (***Support.google.com, 2017***)

SSL – Secure Sockets Layer

SSL is a security technology employed for establishing an encrypted connection between a browser and the server. It has become an industry standard and is used across almost every secure web application online. - ***(Info.ssl.com, 2017)***

2.5.2 –How it was applied

In relation to the employee management web application, numerous security based variants have been applied. An SSL certificate has been purchased and applied to the application, resulting in secure communications between the browser and server. Additionally, the use of HTTPS ensures secure, encrypted transfers.

In compliance with the principles of the Data Protection Act, only relevant details are being collected and stored. The password field is hashed using the php bcrypt function and a salt for

added encryption. Details such as NI numbers are being obtained for the use of the finance department when registering an employee's working status.

2.6 – Conclusion

This literature review has provided a solid understanding and clear insight into the laws and legislations that govern the security and protection of an individual's personal data. Research into the incidents within the industry and the subsequent consequences, has reiterated the importance of adhering to the Data Protection Act.

The research into security based technologies lead to HTTPS and SSL being implemented into the employee management application. Every precaution necessary will be taken, on part of the developer, to ensure the application itself abides by the principles of the Data Protection Act.

3.0 - Requirements Analysis

The gathering and understanding of requirements is a fundamental element in building a successful project. The first 'phase' of most software development methodologies is the requirements analysis. This chapter consists of numerous analysis-based techniques which help derive, prioritise, and understand the requirements. Relating specifically to this project, the following requirements have been discovered through varying stages of the project life-cycle. As AGILE is focussed primarily on iteration-based development, the following analysis techniques have been altered multiple times before achieving the finalised results.

3.1 – Software Development Methodology

Software development methodologies are formulated approaches to constructing a finished piece of functional software. These patterns are based around core principles, of which there are many advantages and disadvantages. The success of these methodologies is based predominantly around the context in which they are applied. Adopting a software development methodology approach is imperative to the successful build of an application, for a variety of reasons. The two main methodologies considered regarding this project were 'Waterfall' and 'Agile' models. Below they are discussed in more detail and a suitable decision is confirmed.

3.1.1 – Waterfall methodology

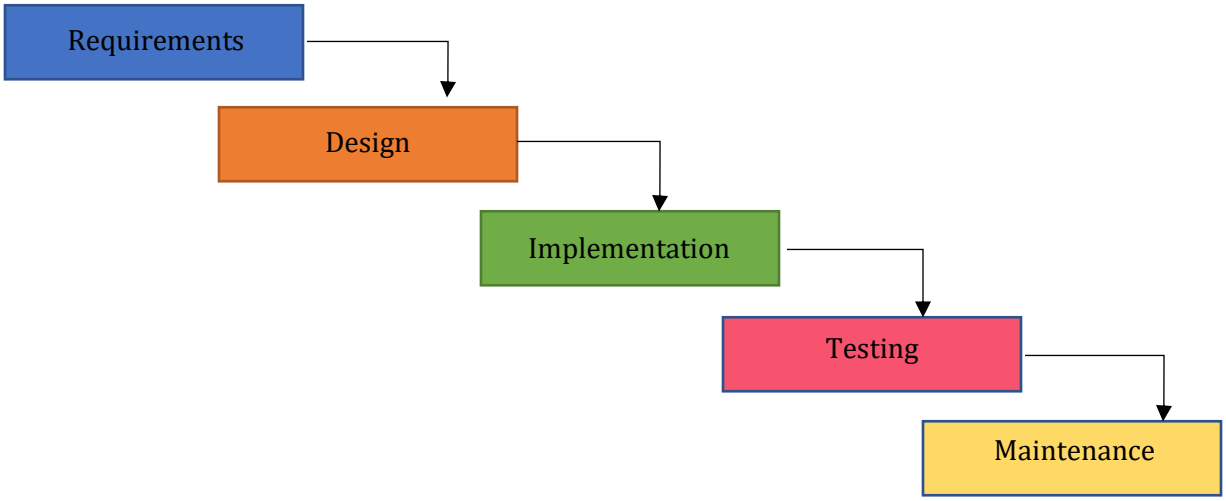


Figure 1 – Waterfall Model

The above diagram represents the 'Waterfall' project development model. This model is known for being sequential and non-iterative. The advantages of employing this type of design process is that each segment of the model will receive a comprehensive level of quality assurance due to the lack of overlap when progressing through. Each section is almost complete before moving onto the next section. However, disadvantages of employing such a method would be the inability to perform alterations through-out the development cycle. With modern development, the relationship between developers and clients is almost continuous and the need for alterations is typically considered a standardised request.

3.1.2 – Agile Methodology

Figure 2 - Agile Model

The term 'Agile' when referring to software development methodologies, is used to broadly describe the way in which several incremental and iterative-oriented models function. A predominant property of Agile is the flexibility in which it provides both the developers and clients. It allows for consistent change and encourages development creativity. In contrast to sequential methodologies, Agile is iteration-based and considers the client as a member of the development team. Deliverables are produced and presented to the client throughout the life-cycle, and future iterations are mainly based from client feedback. There are numerous number of Agile methodologies that implement the core principles, known as; Scrum, Extreme Programming (XP), Dynamic Systems Development Method (DSDM) etc.

3.1.2 – Methodology Comparison

Below is a table detailing how each type of software development methodology responds to typical s/w engineering properties.

Feature	Waterfall	Agile
General Risk	High	Low
Time to implementation	Long	Short
Communication with client	Low	High
Ability to change	Hard	Easy
Time Planning	Long	Short
Client Satisfaction	High	Typically Low(er)

Having considered the above arguments and the benefits of both methodologies, the most beneficial architectural pattern for this project would be 'AGILE'. Typically, Agile approaches incorporate team work into the majority of its principles. As this is an individual effort, core principles such as non-sequential, iterations, and constant communication with the client will be implemented.

3.2 - Stakeholder Analysis

A stakeholder study is a vital aspect of project analysis as it helps visualise and determine how the project is going to be built to cater for each stakeholder. Below is a stakeholder analysis matrix, which contains the investors for the employee management system, and compares both their interest with the power they have over the finalised outcome.

Stakeholder Analysis Matrix

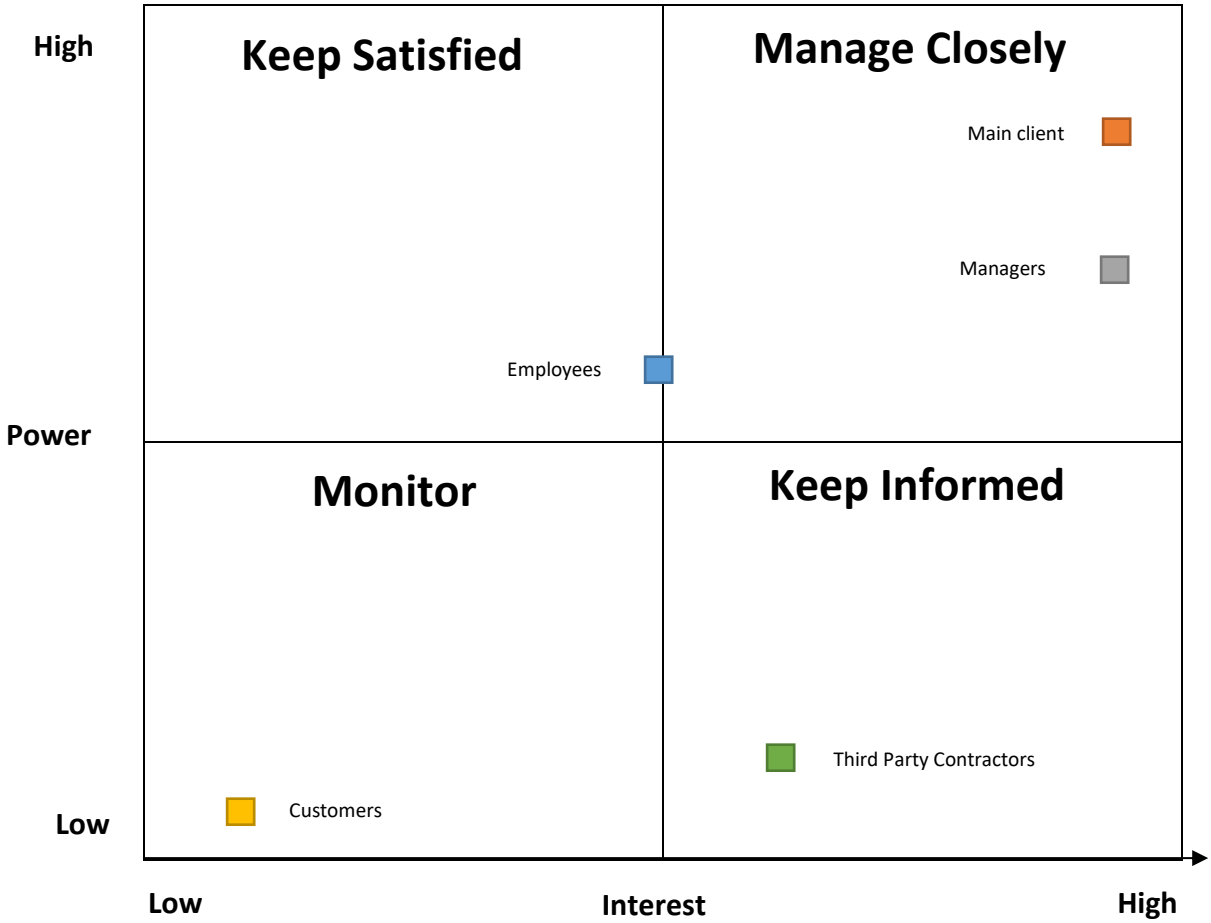


Figure 3 – Stakeholder Analysis

Main Client (CTO)

The main client for this management system is the CTO *(Chief technology officer*) of the company 'Staxoweb'. When viewed as a stakeholder to the project, he has both high interest and power over the development. As the requester and commissioner of the project, he will want to be included and updated throughout the entire process, and able to alter functionality and other properties during this time.

Managers

Managers are considered an important stakeholder as they will be using the system daily. They will be directly involved in the requirements analysis and therefore have high power in the system. The purpose of this system is also to benefit them in their time and resource management with their own internal projects. Although they may not have as much influence over the outcome as the main client, they will shape how specific features function.

Employees
Employees will also have a high interest in the system as they will be using it regularly. They, of course will have some influence over the functionality as they will be undertaking analysis procedures to determine required functionalities. However, they have less power than superior figures within the company. As stakeholders, they will need to be considered and kept satisfied as to avoid user rejection and ensure good usability.

Third Party Contractors
Staxoweb (the client) works closely with third-party web development companies to distribute workload at agreed costs and acknowledgment. The work taken on by Staxoweb will have agreed time deadlines and budget. Therefore, an increase in overall productivity could potentially benefit the third-party company's reputation and overall efficiency. Due to this reasoning, they will hold a high interest in the system, without having a considerable impact on the development.

Customers
Customers may not be aware of the implementation of an improved managerial system within the company, but still need to be considered a significant stakeholder as they may experience an improved service as a direct result of this. As shown on the diagram, they will have little interest and possess minimal influence over the system, other than the fact that the overall goal is to improve productivity throughout the company.

It is extremely beneficial to understand who the system is being built for and consider the multiple parties that can be affected, whether positively or negatively during the process. The visual aid of such an informative diagram regarding stakeholders helps developers understand the globality of the project and reiterates the main goals.

3.3 - SWOT Analysis

A SWOT analysis helps depict positive and negative factors regarding a business/project. Considering both internal and external values, it displays the information in a visual concept to emphasize potential business supporting/damaging factors. This form of analysis is typically used to provide justification for the development of a project.

SWOT Analysis Matrix

Strengths	Weaknesses
✓ The client is extremely dependable and is reliant on a fully functional outcome. ✓ The outcome will help drastically with overall business productivity and management ✓ Potentially profitable to a business deploying successful man-management techniques because of project management service.	✓ The allowance for additional features may invoke pressure to time constraints and increase complexity ✓ Partially reliant on third party services (hosting provider, web servers etc.)
Opportunities	**Threats**
✓ The possibility of additional features and expanding the product. ✓ Gap in the market for multi-component built software regarding project management/communication ✓ Can appeal to audiences un-related to Web solutions (any management/employee based business).	✓ The rapid development of technology may result in additional software development, post release. ✓ Competitors offering similar products for lucrative pricing may force the loss of a client

Table 1 – SWOT Matrix

As shown in Table 1 the strengths and opportunities greatly outweigh the weaknesses and threats. The negative aspects both internally and externally are of a fairly low risk and would be categorised as pre-cautionary aspects to consider. Completing a SWOT analysis aids in the justification of the project, and consulting with the above table, this project is more than viable to be successful.

3.4 – User Stories

A User story is a simple, short description of carrying out a function from the user's perspective. User stories are often drafted by the developer and/or the clients, to cement the understanding of how the system reacts in certain situations. This analytical technique also aids the deriving of functional and non-functional requirements, and clarifies lingering ambiguity between both parties. The following user stories were created due to mid-development meetings with the client's to resolve project functionality queries.

3.4.1 – Employees

"***As an employee of Staxoweb Ltd, I want to be able to submit requests for holiday and overtime to my relevant manager. I should be able to view all current sent requests and their status, so that I'm clear on my current situations***".

"***As an employee of Staxoweb Ltd, I should be able to view all assigned tasks given to me from my dashboard. These tasks should be listed in terms of priority (deadline) and I should be able to click begin when I work, and end when I stop***".

3.4.2 – Managers

"***As a manager of Staxoweb Ltd, I want to be able to view all up-to-date employee requests from the same interface. I should be able to flick simultaneously through each, view their current status and either confirm or decline from here***".

"***As a manager of Staxoweb Ltd, I should be able to add new projects with the relevant fields. I should also be able to add deliverables to these projects. Then I should be able to add and assign tasks to employees through these tasks***".

These user stories in particular were poignant to focus on, as the original iteration of the application did not incorporate these elements due to requirement ambiguity. Once speaking with the client and creating these user-stories, it became extremely apparent about how the project/deliverable/task assignment and measurement was going to function.

3.4 - Use-Cases

A Use-Case diagram is a modelling procedure to help identify the users of the system and their actions to achieve the overall aim. They are used to visualise how the user interacts with the system, and to obtain requirements. Below are Use-Case diagrams to represent the users' actions when performing essential requirements included within this system.

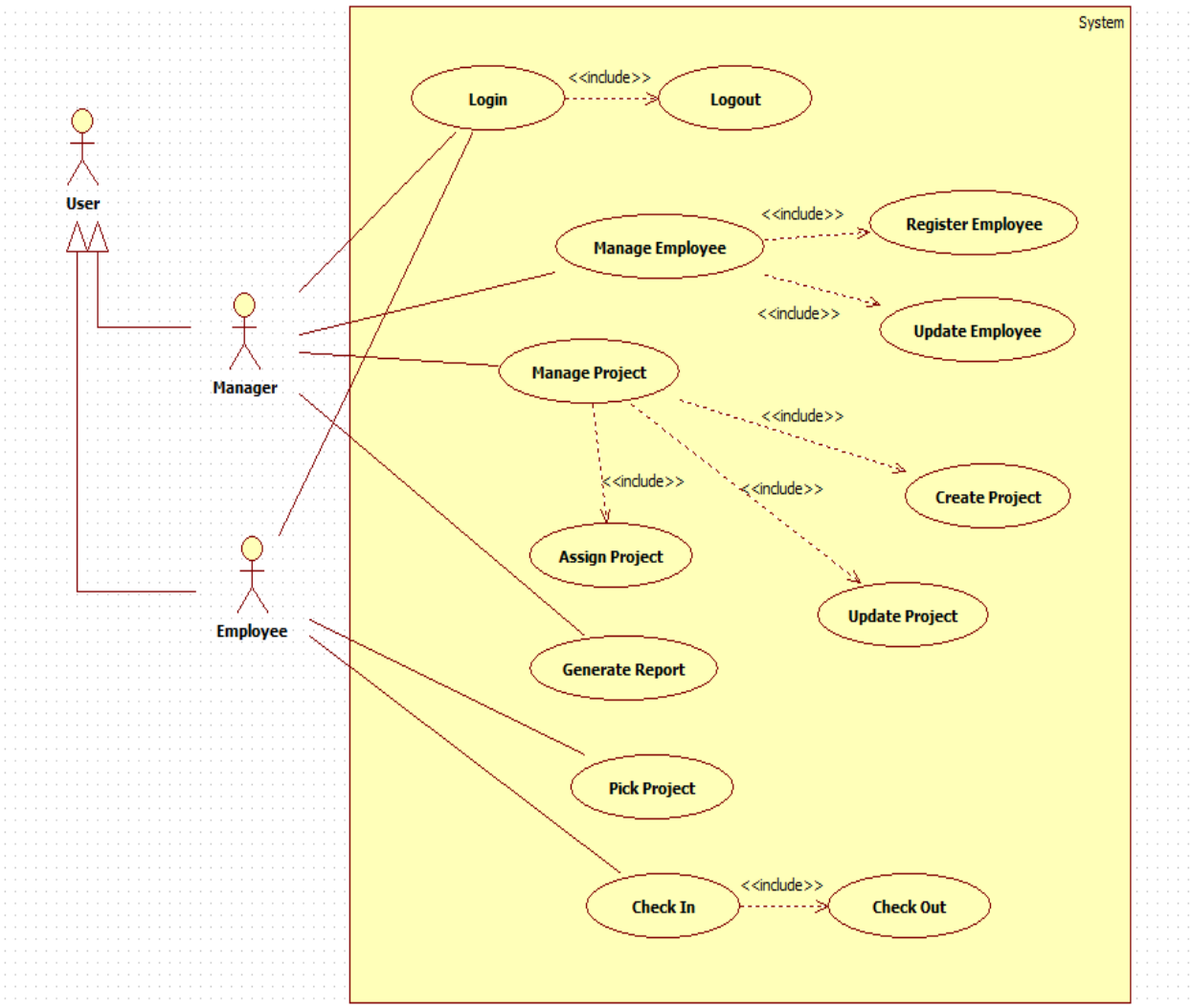


Figure 4 – Login Use-Case Diagram

Referring to ***Figure 4***

Above is a Use-Case diagram for the action, 'login'. It displays how each user logs into the system and the post functionality of successfully doing so. As each user is greeted with a unique interface, the functionality process will differ slightly. The 'Manager' actor has access to functionality such as manage projects, of which contains sub-functionalities such as creating, updating, and deleting projects. The Manager is also able to manage specific actions regarding employees, such as creating, deleting and editing. The employee's functionality within this system boundary is the ability to check in and out for attendance tracking purposes, and pick tasks/projects from a list assigned from their respective manager.

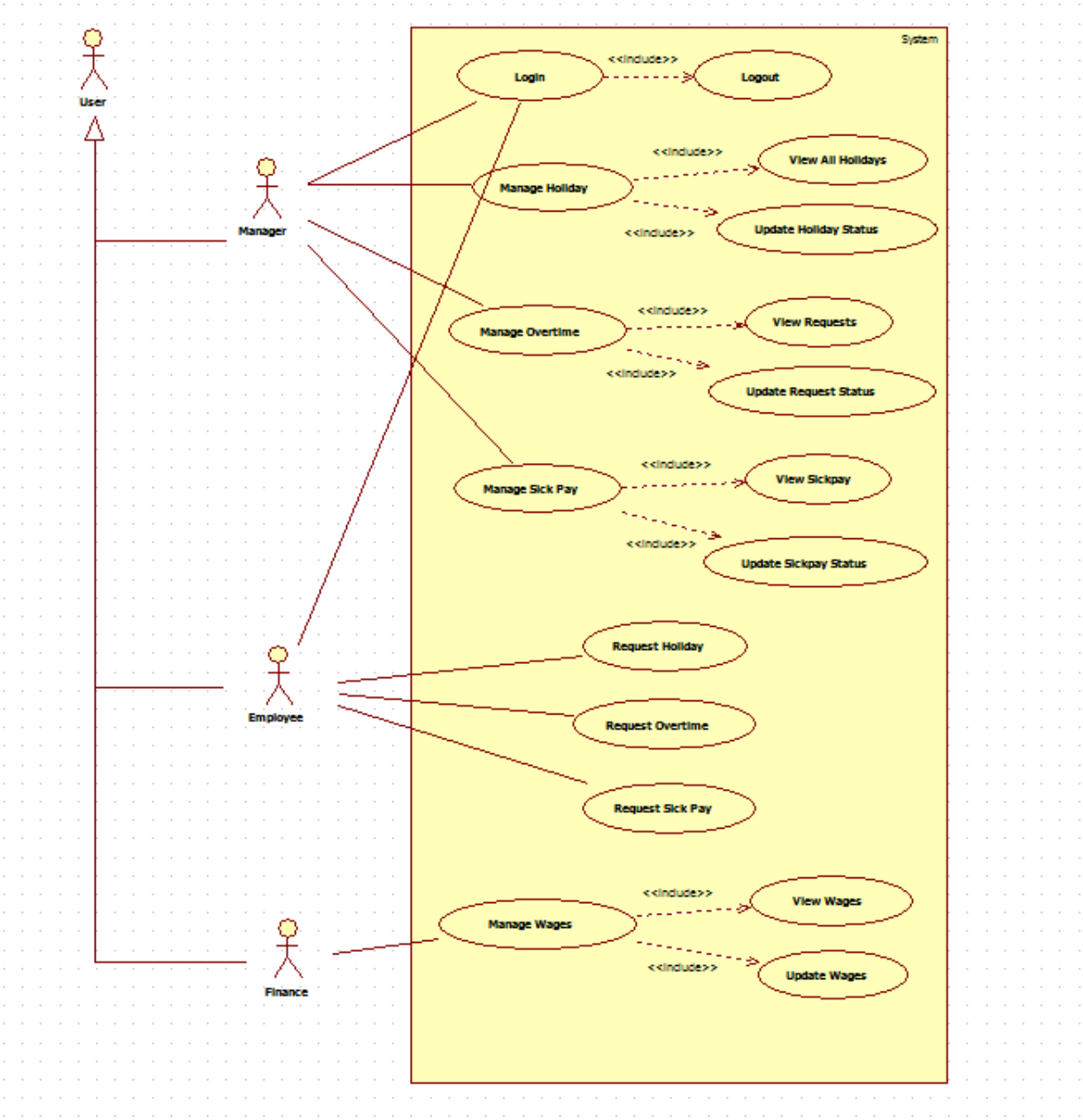


Figure 5 – User Privileges Use-Case Diagram

Referring to ***Figure 5***

This Use-Case diagram represents the user sending a request and how that action impacts the system use-process of the corresponding manager. As shown above, once the 'Employee' actor logs into the system, they are able to submit holiday, overtime, and sick pay requests. These requests are sent to their assigned Manager. Once the 'Manager' actor logs in, they are alerted with a notification of the outstanding request. They have multiple 'manage' sections in which they can alter the status of the request.

3.3.1 - Actors

The users of the system are referred to as 'Actors' when concerning Use-Case diagrams. These are represented as 'stick-man' icons within the diagrams, typically located outside of the 'system' boundary. Each of these actors will have different processes and privileges, as detailed below.

Administrator

The Administrator will possess the ability to log into the system and create departments. Each department will consist of managers and employees. Additionally, they will be able to assign each manager to the relevant department and each employee to the relevant manager.

Manager

The manager has access to the following functionality:

- Able to create new Employees to their team
- Accept or Decline Employee requests
- Generate Attendance reports
- Generate Project/time reports for each Employee.

Employee

The Employee user type will have access to the following functionality:

- Able to choose a project
- Check in / Check Out
- Create a Holiday, Overtime or Sick pay request
- Generate a monthly report of their wages based on attendance.

Finance

The Finance Actor has the following system privileges:

- Able to log into the system and view all Employees monthly wages
- Able to manually alter employee's wage if required (exceptional circumstances)

Identifying the users of the system is extremely important when considering functional requirements. Especially when relating to an employee management system, as each 'actor' will have varying interfaces with privileges to different functionality.

3.5 – Use-Case Scenarios

Use-Case scenarios are an instance of typical situations that would be encountered as a user of the system. These scenarios are divided into sequential processes of how the application is expected to react to each stage. This analytical technique helps clarify how the system is anticipated to function, and aids the developer and client in the understanding of certain processes.

ID	1	
Name	User Login	
Summary	The following steps are required for the User to successfully log into the system and reach their relevant user interface	
Priority	Very High	
Preconditions	None	
Post conditions	The user is now logged into the system	
Primary Actor(s)	Manager, Employee, Administrator	
Secondary Actor(s)	None	
Trigger	User can operate the system.	
Main Scenario	**1**	The system prompts the user for username/email and password. A log in button is included.
	2	The user enters the relevant details and clicks log in.
	3	The system authenticates the login details and the user is confronted with the relevant interface
Extensions	3a – If the user details are incorrect, the system will display an error message detailing the invalid fields. 3b – If login attempts exceed 5, the account will be locked and an administrator must be contacted.	

Table 2 – User Login Use Case Scenario

ID	2
Name	Add Employee
Summary	The following steps are required for the Administrator/Manager to successfully add an employee to the system
Priority	Very High
Preconditions	The completion of Use-Case scenario id 1
Post conditions	The employee is now added to the system and assigned to the correct manager

Primary Actor(s)	Manager, Administrator	
Secondary Actor(s)	Employee	
Trigger	Manager can add Employee to the system.	
Main Scenario	**1**	The interface displays an employee related section with a button for 'add new employee'
	2	The user clicks on the button
	3	The system prompts the user with input fields regarding the employee's personal information.
	4	The user clicks on the submit button
	5	The employee is added to the system
Extensions	5a – If any invalid data is entered, the system will display an error message detailing the point of invalidation.	

Table 3 – Add Employee - Use Case Scenario

ID	3	
Name	Employee Request	
Summary	The following steps are required for the Employee to successfully send a request to their Manager for approval	
Priority	Very High	
Preconditions	The completion of Use-Case scenario id 1	
Post conditions	The request has been sent to the manager	
Primary Actor(s)	Employee	
Secondary Actor(s)	Manager	
Trigger	Manager has received request and can update the status.	
Main Scenario	**1**	The system presents the employee with holiday, overtime, and sick pay sections
	2	The user clicks on the relevant section

	3	The system will present the employee with the option to send a new request.
	4	The employee clicks on this option
	5	The system presents the user with input fields and calendar fields for date range.
	6	The user clicks the submit button and the request is sent to the relevant manager.
Extensions	6a – If any of the input fields are invalid, the system will display an error message detailing the invalid fields. 6b – If the requested times conflict with 'blocked dates' set by the manager, the system will display an error message stating the requested dates are blocked.	

Table 4 – Employee Request - Use Case Scenario

As shown in the above Use-Case scenarios, extensive information such as the conditions, actors, triggers, and steps are detailed for each specific situation. Having a firm grasp on how the system is expected to behave is an extremely reliable method of extracting functional requirements. Having these scenarios validated by the client also ensures that both client and developer are working towards the same result.

3.6 - Functional & Non-Functional Requirements

From executing the analytic techniques within this chapter, and conversing extensively with the client, functional and non-functional requirements have been derived and are listed below.

The functional requirements are features that must be included and operate correctly within the system to achieve the intended functionality. The following requirements have been verified by the client as the 'main' features to be contained within the application.

3.6.1 - Functional Requirements

#	Functional Requirement
FR1	Login with appropriate Dashboard
FR 2	Generate Attendance Report
FR 2a	Generate Project/Time Report
FR 3	Create Requests ***(as an employee)***
FR 3a	Alter request status ***(as a manager)***
FR 4	Add new Project ***(as a manager)***
FR 4a	Add new Deliverable ***(as a manager)***
FR 4b	Add new Task ***(as a manager)***
FR 5	Assign Deliverable ***(as a manager)***
FR 5a	Assign Task ***(as a manager)***
FR 6	Customise dashboard appearance ***(as an administrator)***

FR 7	Set Employee Salary
FR 8	Add New Employee ***(as a manager)***
FR 9	Generate Employee Monthly Salary
FR 10	Notify if Employee is absent
FR 11	Live message between Employee & Manager
FR 12	Generate Financial Report
FR 13	Generate PDF Invoices

Table 5 – Functional Requirements List

3.6.2 - Non-Functional Requirements

The non-functional requirements are essentially the remaining features that are not included within the functional requirements. They focus predominantly on the inclusive operation of the application, rather than the specific behaviours.

#	Non-Functional Requirement
1	System must log in and present the correct dashboard within 10 seconds of submitting.
2	The system must have a user-friendly interface across multiple devices and platforms
2a	The system must be fully responsive
3	Relevant user data must be encrypted and the system should be secure
3a	The system must adhere to the Data Protection Act
4	The system must be readily available (at least 364 days a year)
4a	Main system functionality should still be able to perform in the case of a fault.
5	Data constraints must be set in order to ensure data integrity

These requirements were derived from the techniques and processes shown within this chapter. The consistent feedback from the client and iterative-based development style (typical of AGILE) allowed for this extensive/agreed list to be constructed. The list displayed is the settled list for the deliverable date as agreed with the client.

3.6.3 - MoSCoW

The ability to successfully deliver a project relies on the developer's understanding of the user requirements. To help cement their understanding of the requirements, priority oriented methodologies are implemented.

In this case, the MoSCoW analysis procedure is being employed. MoSCoW is typically used to prioritise overall system requirements. MoSCoW is an acronym for Must, Should, Could and Would.

M – Must Have
Requirements falling under the must-have category of the prioritisation method are necessities for the system to be considered a success. These requirements must be implemented as agreed with the Stakeholders.

S – Should Have
Requirements within the 'should-have' category are considered important but not vital to the delivery of the project within the agreed timeframe. These requirements can be further discussed with Stakeholders and agreed to be implemented at a later date.

C – Could Have

Requirements categorised as 'could-have' are desirable features of the system, but not necessary. These requirements are often included if not impacted by time and resource constraints as a method of increasing client satisfaction.

W – Would Have
Requirements considered as 'Would-have' are the lowest prioritised for that moment in time. Stakeholders have agreed these requirements to be included as an extension of the submitted project.

The prioritization of the requirements undergone in the MoSCoW analysis have been agreed by the relevant stakeholders. By prioritising requirements, the developer can work towards delivering the most important features required for the project.

Refer to ***Table 6.0*** for the prioritised requirements list.

MoSCoW Prioritisation		
Project	Bespoke web-based employee management application	
Constraints	**Time**	**Budget**
	8 months	£100
M	Login with appropriate Dashboard Generate Attendance Report Generate Project/Time Report Create Requests ***(as an employee)*** Alter request status ***(as a manager)*** Add new Project ***(as a manager)*** Add new Deliverable ***(as a manager)*** Add new Task ***(as a manager)*** Assign Deliverable ***(as a manager)*** Assign Task ***(as a manager)*** Set Employee Salary ***(as a manager)***	
S	Customise dashboard appearance ***(as an administrator)*** Notify when employee is absent	
C	Generate Employee Monthly Salary Live messaging between Employee & Manager	
W	A mobile application for use on the move Generate Financial Report Generate PDF-formatted invoices	

Table 6 – Prioritized Functional Requirements List

3.7 - Discussion

The practices exhibited during this chapter were executed numerous times during development to acquire and include the updated requirements derived from the client. The completed list of requirements were shown to the client, and the MoSCoW list was constructed based upon their input of prioritisation, relating realistically to time and budget.

The users gathered from the user-stories were questioned by the client. They emphasized that the assumption of a 'Finance' user type being included was incorrect, despite finance-related functionality being present. The client explained that the financial data attained from the system would have to be manually examined by a manager or administrator to validate the accuracy. This is a precautionary method within the bedding-in period to ensure minimal detrimental imprecisions are discovered.

After considering the completed requirements list, and the time/budget constraints, I am confident as the developer, that the finalised product can be produced on time with all requested functionality included (Assuming no additional time constraints in design chapter). The client has stated, the characteristics included within the latter end of the prioritisation list are to be included in a future iteration/version beyond the agreed deadline.

4.0 - Design Chapter

4.1 - Introduction

Before the implementation of a project can begin, detailed design stages are required to be carried out to help visualise and fully understand how the finalized product is going to function. Many properties must be considered. These properties can be split into two main phases; User experience and Data Modelling.

User Experience delves into the visual element of the project, and challenges the developer to create interesting and effective ways to display information to the user. Data Modelling is exploring how the data is related, and considers how the transmission of information can be effectively implemented. During this chapter, various design-oriented methods applied within this project will be discussed and validated, through comprehensive dialogue and argument, to produce the most effective and beneficial foundations to begin implementation.

4.2 - Data Modelling

4.2.1 - Initial Entity Relational Diagram

The construction of an entity relational diagram is dependant predominantly on the functional requirements derived from the client. An ERD demonstrates how the database is structured and helps envisage the interaction between tables in terms of the transmitting and storing of relevant data. Below is the initial entity relational diagram created solely from the functional requirements obtained from an early stage of development.

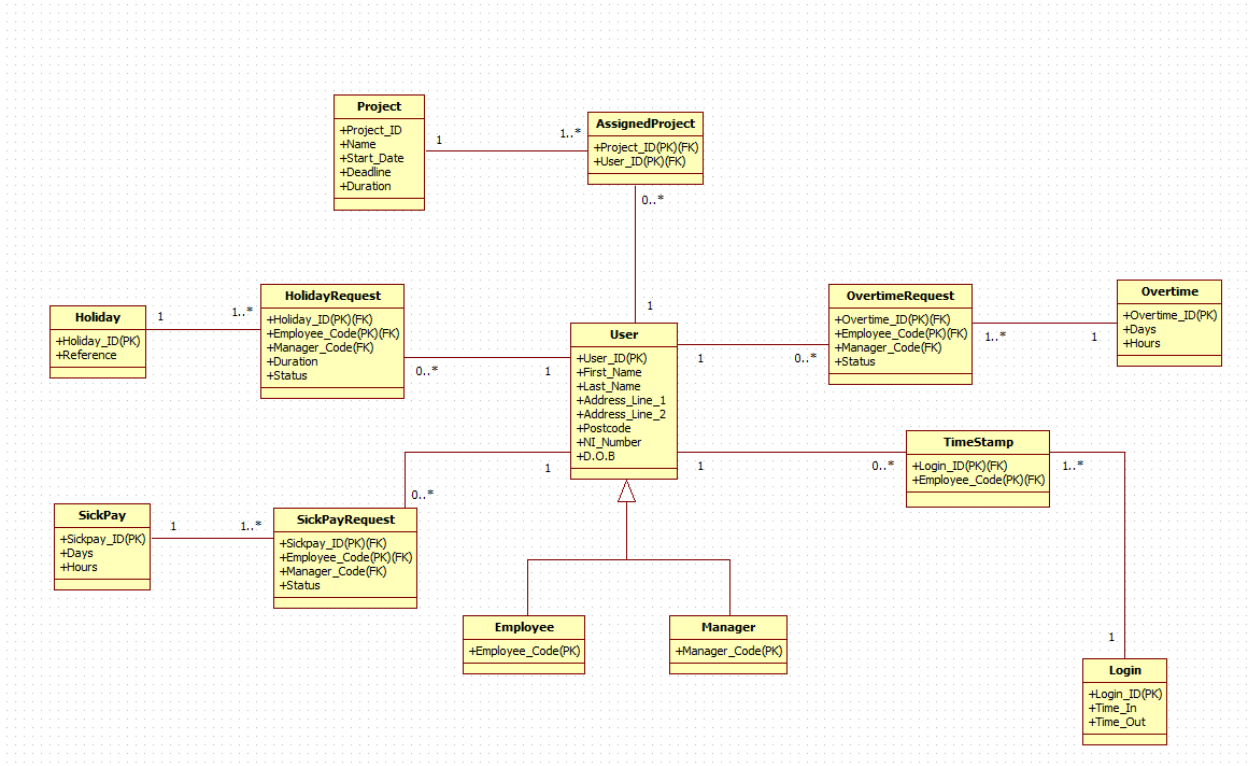


Figure 6 - Initial Entity Relational Diagram

This diagram contains 13 tables and displays how they're going to interact to achieve the correct functionality. Within each table are fields related to the type of information being stored.

The bolder arrow linking 'Employee' and 'Manager' to the 'User' table represents inheritance. This is due to the common attributes within both types of users, which ensures reliability and efficiency when building the database. The numbers located at the base of each relationship is known as 'cardinality'. This signifies the 'type' of relation between each table and enforces rules such as 'must have' or 'may have'. A rule of thumb within database design is that no relationship between tables can be a 'many-to-many' relationship.

During the presentation of this diagram to the client, overlooked redundancies began to surface in terms of desired functionality on their part. The explanation of the relationship between tables and the type of data being stored resulted in additional functional requirements being derived. A common feature within AGILE software development is the re-visiting of previous processes and the ability to alter development. This example emphasizes the importance of a well-structured design procedure.

Please see below the revised entity relational diagram assembled, based on client feedback.

4.2.2 - Revised Entity Relational Diagrams

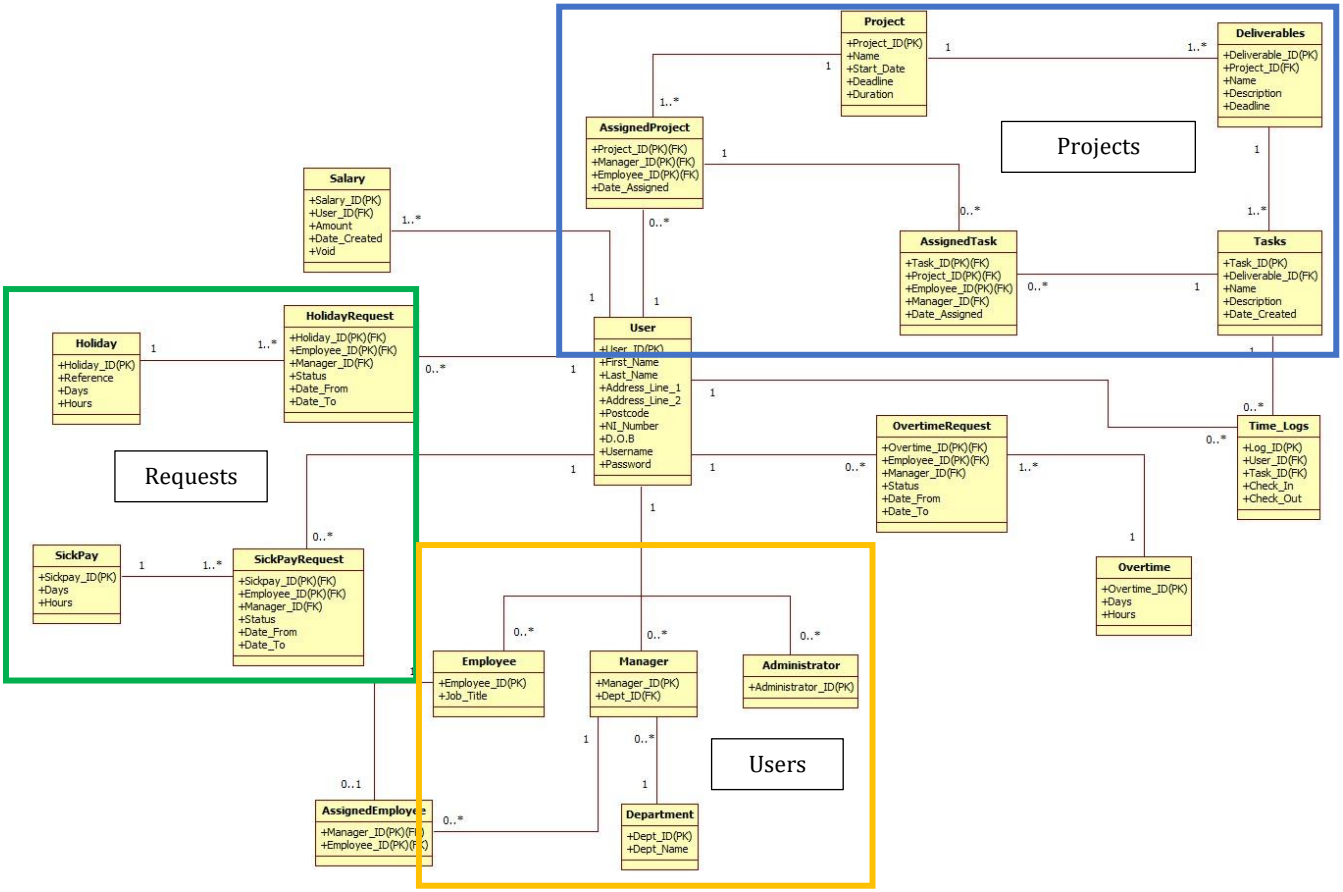


Figure 7 – Revised Entity Relational Diagram

Referring to ***Figure 7***
As shown above, the revised entity relational diagram is far more complex than the initial ERD. The additional tables are the result of a consultation with the client in which an explanation of the initial ERD took place. Once this meeting was underway, it became apparent there was some ambiguity between the client and myself in terms of how the system was going to function. From this meeting, a whole new feature was derived and in-turn, created further functional requirements.

This feature was the extension of the project management related side of the system. The client requested that each assigned project be broken down into deliverables, of which consist of tasks. As the time-tracking feature primarily records the time against each project. The addition of this feature should not impact any other 'main' project feature, so long as task is somehow related back to the project. Realistically, the deadline for the system should not be under too much strain with the addition of this functionality.

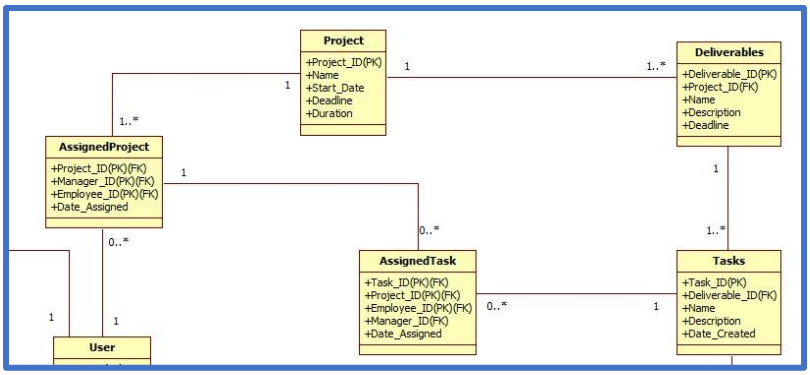


Figure 8 - Project/Deliverable/Task Relationship

Referring to ***Figure 8***
The section above is located on the right-hand side of the revised ERD, and is detailing the relation of tables within the database regarding efficiently implementing the project/deliverable/task timing functionality. The flow of relation is indicated stemming from the user. The users have a direct relation with the assigned project table, in that the manager and employee ID attribute is required to perform this functionality. Following the sequence across the top of the diagram, you can see that project is related to the assigned project table due to the project ID attribute, and deliverables are attached to project through the same attribute foreign key.

By using this connecting method, it allows for assigned employees to view their tasks via deliverable and project. The resulting functionality is that the manager can assign a project, set a deliverable and tasks, and assign them to employees which can then view and complete them. The timing functionality comes into play by joining onto the tasks table via the Task ID attribute. The employee will be required to click a button when starting and finishing which will create a timestamp record of both. By using php, the calculation will be able to determine the overall amount of time spent, and can be displayed as a visual graph in the reports view.

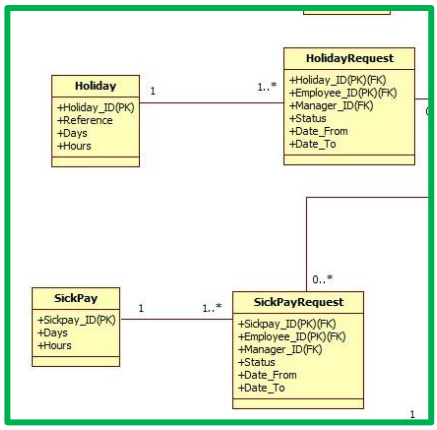


Figure 9 – Request Functionality

Referring to **Figure 9**

The above segment represents the employee request functionality. The system will be able to process requests for holiday and overtime. The requests are handled through the 'joining' tables between the user and the requested feature. These request tables will contain foreign keys from the manager and employee table to ensure the requests are sent to the corresponding manager, and from the correct employee. All request features contained within this application will function using this data architecture.

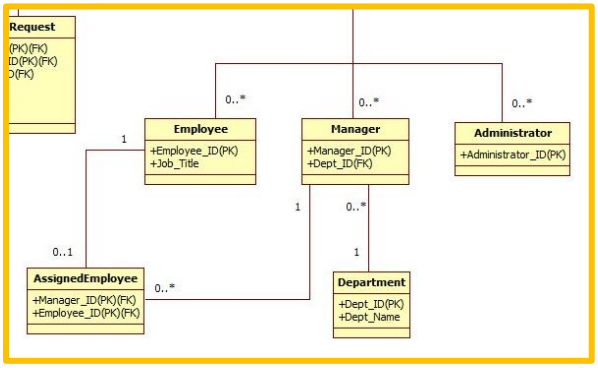


Figure 10 – User Type Defining

Referring to **Figure 10**

This figure is referring to the relationship between the user table, and how different users are defined. Whenever a user is created using the application, the data query will insert a record into both the user table and the corresponding user type table. Within the user table, there is a field labelled 'user_type' which defines the type of record being entered, and in turn will enter another

record into the relating table. These user type tables contain predominantly their unique primary key and the User ID foreign key to receive all the relevant information and display it on the view. This will also allow for the defining of features available to specific users, using conditional logic in php.

4.2.3 - Activity Diagrams

To fully comprehend how the designs are going to support required functionality, activity diagrams are constructed. These guide the user through specific processes involved in achieving the desired result. It also displays how the system is going to react in terms of different conditions, for example if a null value was present at a vital stage of a process. Producing activity diagrams further strengthens the understanding of the system as a whole, and aids massively when it comes to implementation.

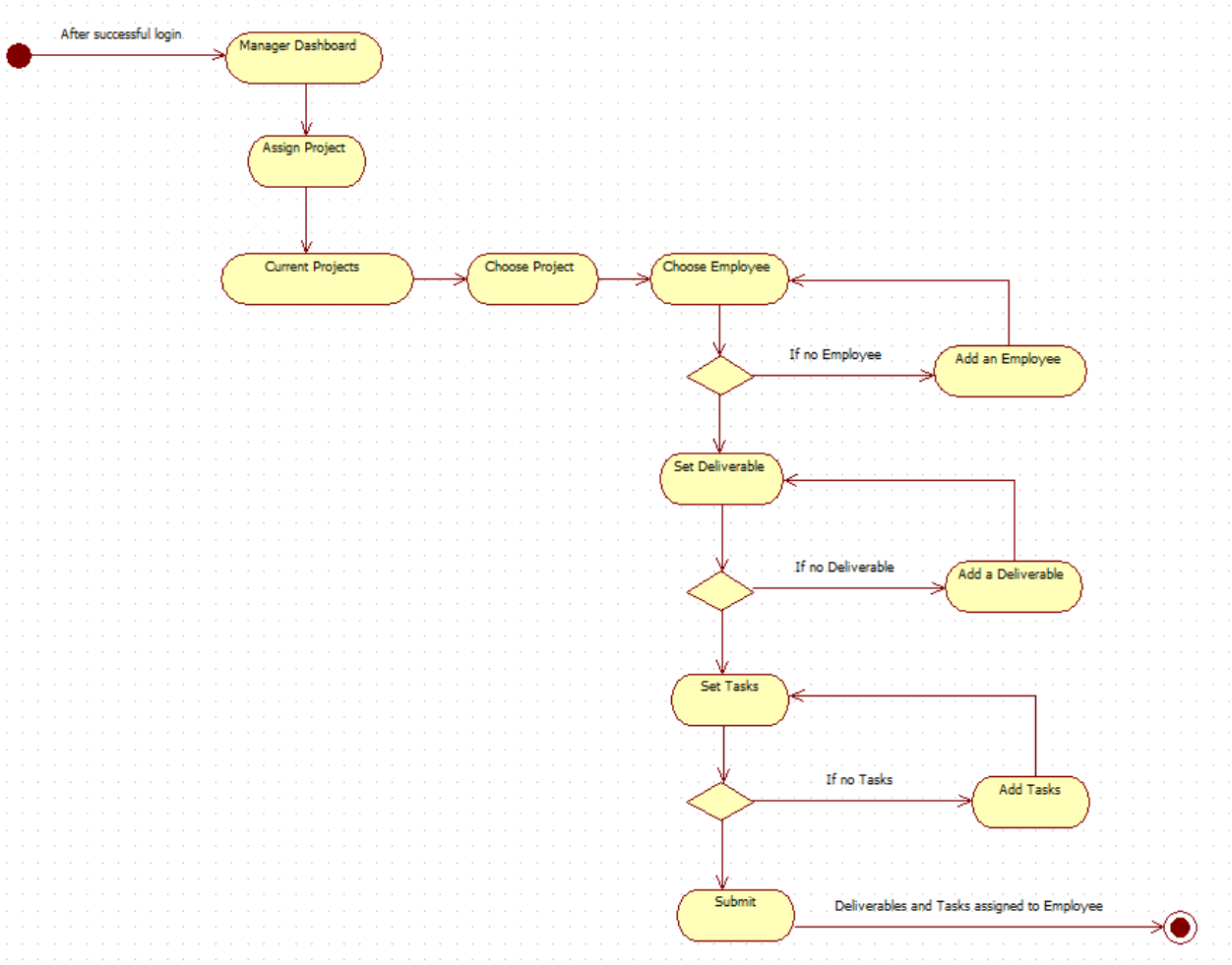


Figure 11 – Setting Tasks Activity Diagram

Referring to ***Figure 11***

This activity diagram is presenting the process undergone by the system/user when a 'Manager' wishes to set a task and assign it to an employee. The diamond shaped sections within this diagram represent a choice. The first choice during this process is to choose an employee for which the task is going to be assigned. If there is no employee set, the system will prompt to add an employee, else the system will continue onto the next stage of the process. Constructing these diagrams is imperative in understanding the requirements of a system, and perhaps discovering additional requirements also.

Table 8.1 – Sending & Receiving Requests

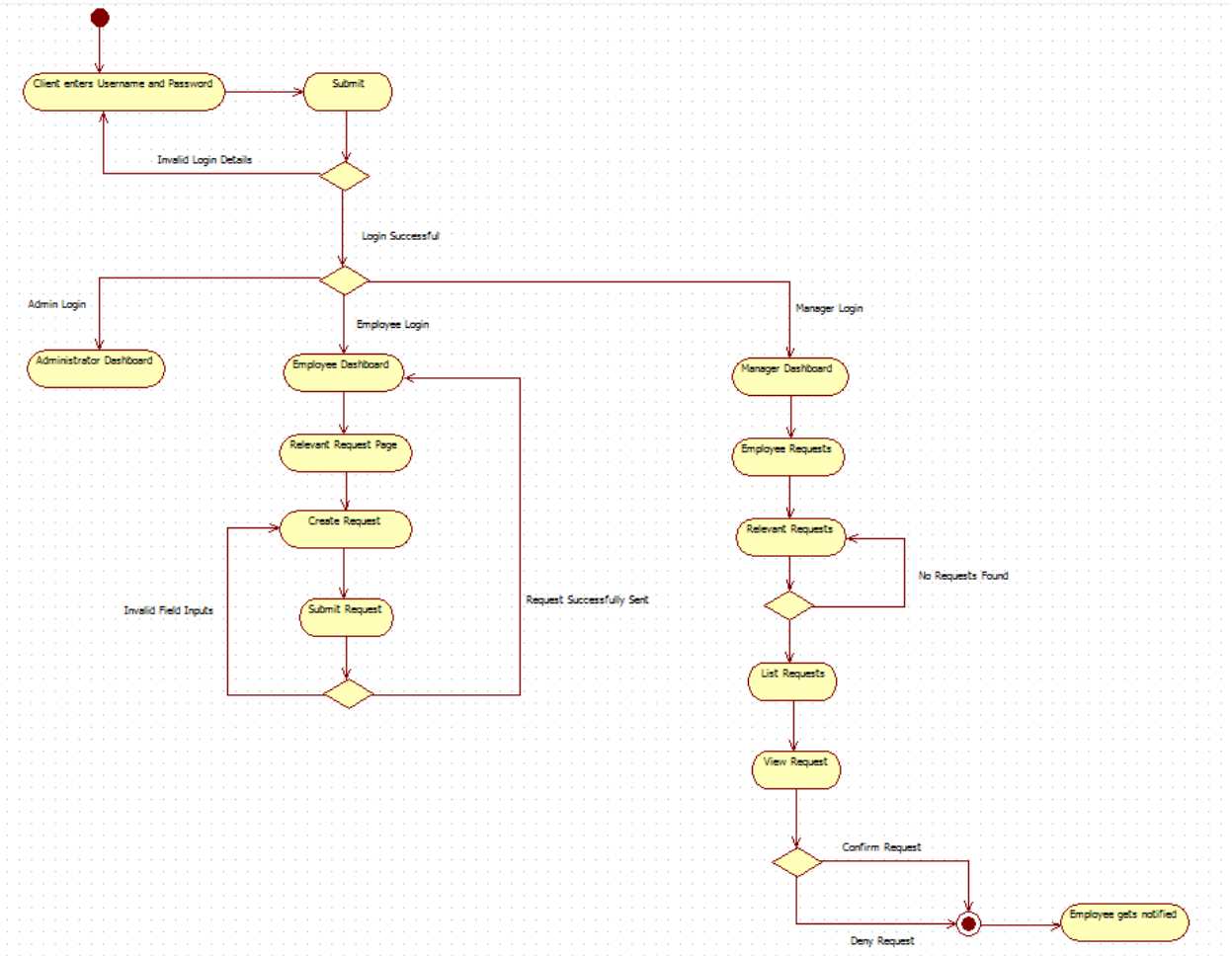


Figure 12 – User Dashboards Activity Diagram

This activity diagram is presenting the process undergone by the system/user when an Employee creates a request, and the manager responds to it. As the processes for both users differ, these could have been split into separate activity diagrams, but as the login process is the same, and the functionality is linked, it can be displayed as one diagram. This diagram emphasizes the need for error-handling when certain aspects are accessed with null values. For example, if no requests currently exist, the system needs to be able to alert the correct information to the user.

4.2.4 - Sequence Diagrams

A Sequence diagram is a visual construction/representation of the methods that are undergone throughout a system to perform requested functionality. These actions are arranged in the order of time. It aids the developer to understand how the system is going to handle features, and provides clarity on definite decision making during preparation for the implementation.

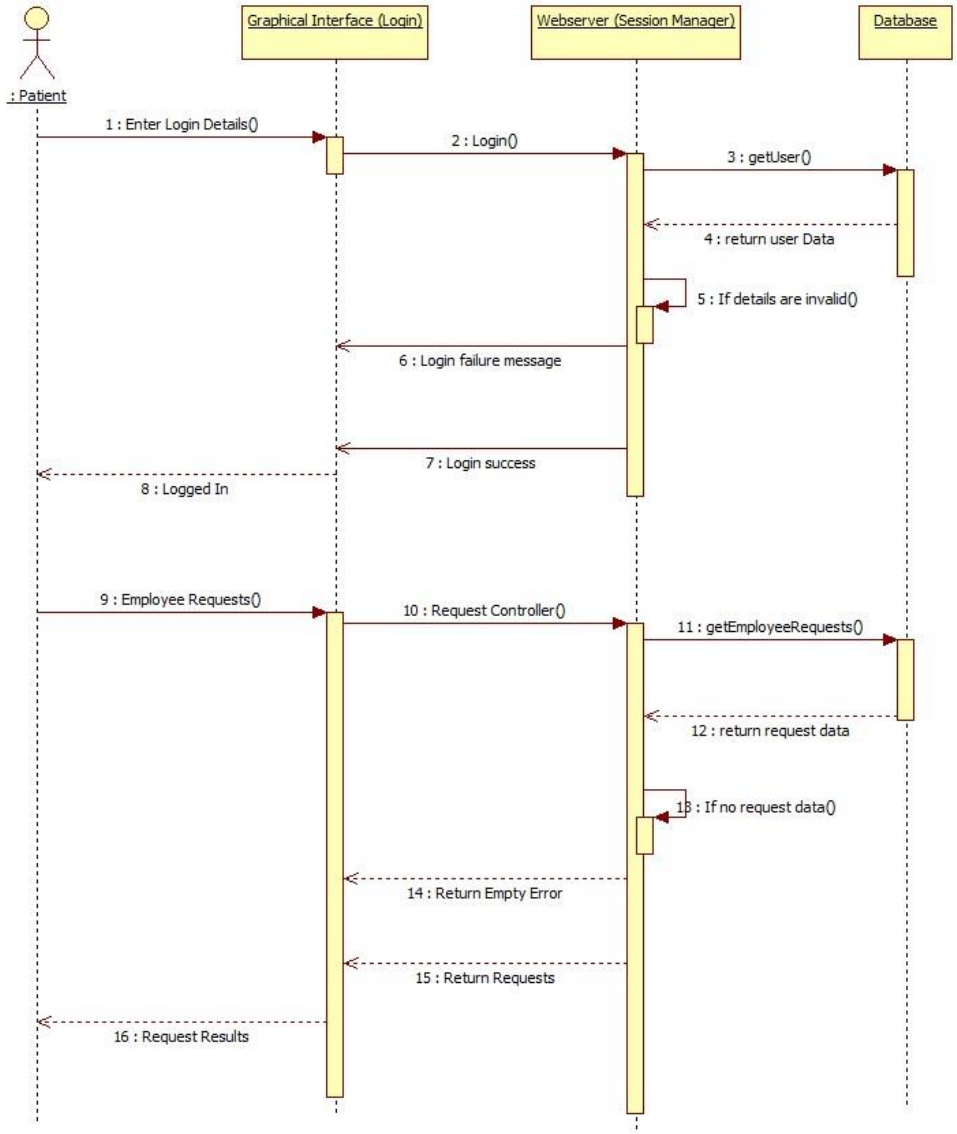


Figure 13 – Login & Employee Request Sequence Diagram

This sequence diagram is showing the flow of information when a manager logs into the system, and views employees requests. It is vital to understand how specific parts of the application are going to interact to achieve this operation. As shown above, when the manager interacts with the login screen, a request is sent to the webserver with the input fields. The webserver then requests information from the database. The returned data is then compared with that of the users input fields, and either grants login via a redirect or provides an error message to the view.

4.2.5 - Data Dictionary

A 'Data Dictionary' is a repository of the information included within the database. It is an extension of the material derived from the Entity Relational Diagram, containing key data such as data types, constraints, unique fields, and any additional notations. It also cements the relational logic between tables to the developer, and allows for a more efficiently structured DB implementation plan.

Table	Attribute	Data Type	Length	PK/FK	NOT NULL	Constraints
Users	User_ID	INT	50	PK	✓	
	FirstName	VARCHAR2	100		✓	
	LastName	VARCHAR2	100		✓	
	AddressLine1	VARCHAR2	100		✓	
	AddressLine2	VARCHAR2	100			
	Postcode	VARCHAR2	10		✓	
	NINumber	VARCHAR2	11		✓	
	DOB	DATE	8		✓	
	Username	VARCHAR2	50		✓	
	Password	VARCHAR2	50		✓	
	UserType	INT	1		✓	
Projects	ProjectID	INT	50	PK	✓	
	Name	VARCHAR2	100		✓	
	StartDate	DATE	8		✓	
	Deadline	DATE	8		✓	
	Duration	INT	100		✓	
Deliverables	DeliverableID	INT	50	PK	✓	
	ProjectID	INT	50	FK	✓	
	Name	VARCHAR2	100		✓	
	Description	VARCHAR2	255		✓	
	Deadline	DATE	8		✓	
Tasks	TaskID	INT	50	PK	✓	
	DeliverableID	INT	50	FK	✓	
	Name	VARCHAR2	100		✓	
	Description	VARCHAR2	255		✓	
	DateCreated	DATE			✓	
	StartDate	DATE			✓	
	EndDate	DATE			✓	

AssignedProject	ProjectID	INT	50	PK/FK	✓	
	ManagerID	INT	50	PK/FK	✓	
	EmployeeID	INT	50	PK/FK	✓	
	DateAssigned	DATE	8		✓	
AssignedTask	TaskID	INT	50	PK/FK	✓	
	ProjectID	INT	50	PK/FK	✓	
	EmployeeID	INT	50	PK/FK	✓	
	ManagerID	INT	50	FK	✓	
	DateAssigned	DATE	8		✓	
Employee	EmployeeID	INT	50	PK	✓	
	UserID	INT	50	FK	✓	
	JobTitle	VARCHAR2	25		✓	
Manager	ManagerID	INT	50	PK	✓	
	UserID	INT	50	FK	✓	
	DeptID	INT	5		✓	
Clockings	ClockID	INT	50	PK	✓	
	UserID	INT	50	FK	✓	
	InOut	INT	1		✓	
	Reference	VARCHAR2	255			
	DTS	DATETIME			✓	
TaskLogs	LogID	INT	50	PK	✓	
	UserID	INT	50	FK	✓	
	TaskID	INT	50	FK	✓	
	CheckIn	DATETIME			✓	
	CheckOut	DATETIME			✓	
Department	DeptID	INT	5	PK	✓	
	Name	VARCHAR2	100		✓	
AssignedEmployee	EmployeeID	INT	50	PK/FK	✓	
	ManagerID	INT	50	PK/FK	✓	
HolidayRequest	HolidayReqID	INT	50	PK	✓	
	EmployeeID	INT	50		✓	
	ManagerID	INT	50		✓	
	Status	INT	1		✓	
	DateFrom	DATE	8		✓	
	DateTo	DATE	8		✓	
Holiday	HolidayID	INT	50		✓	
	HolidayReqID	INT	50	FK	✓	
	Days	INT	50		✓	

Table 7 – Data Dictionary

4.3 - UX & Interfaces

4.3.1 - Wireframes

Wireframes are basic graphic layouts created to aid in the visualisation of the completed interfaces. Many aspects need to be considered when building wireframes, such as usability and client satisfaction. During the construction of the following wireframes, extensive conversations, and thorough discussion occurred between myself and the client. They were presented with early versions of wireframes and the following wireframes were built based upon their feedback.

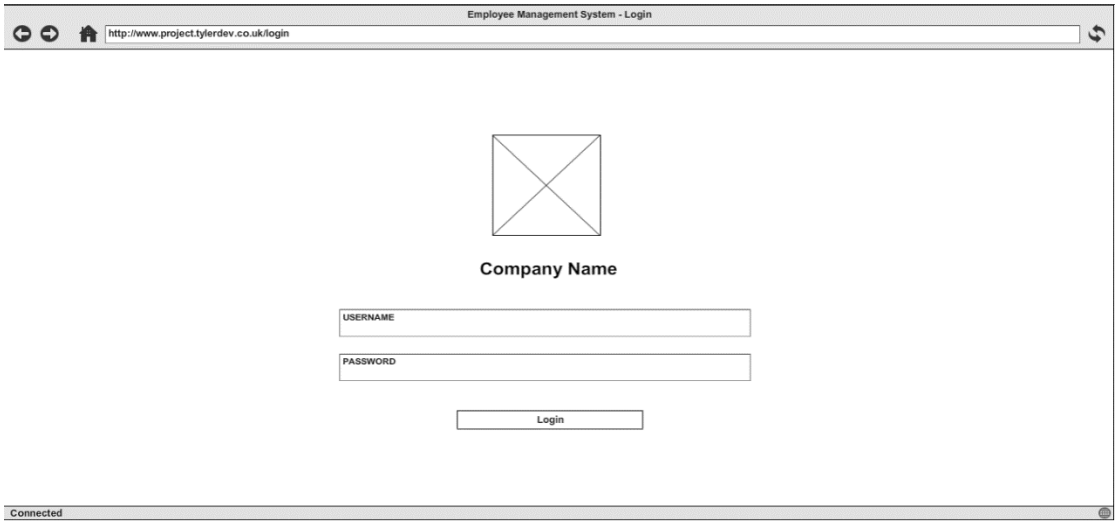


Figure 14 – Login Wireframe

The login page will be the first page the user is presented with once accessing the site. The client requested that the site look professional, and somewhat brand-specific. This page will include the client's company logo, name, and relevant input fields required to access the user dashboard. All external features of this page, including the name, logo and background image will be customisable by the administrator.

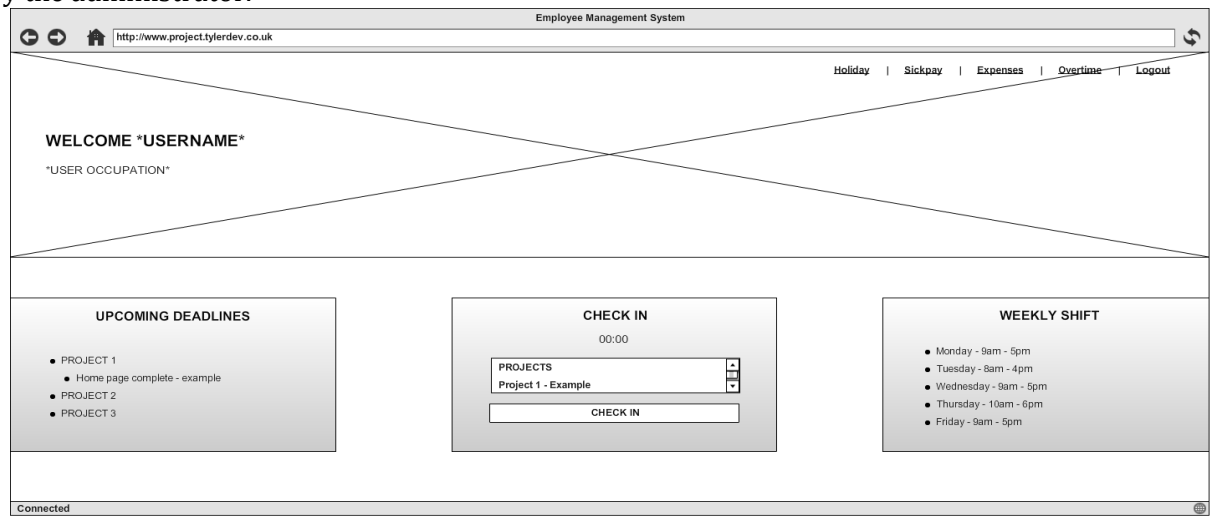


Figure 15 – Employee Dashboard

The above wireframe displays the layout of the page the user would be presented with, if they were an 'Employee'. As shown above, this includes several of the features specified within the functional requirements list. The user will choose a task from the list and click 'check in'. This will record the attendance of the employee as well as 'tracking' the time against the project. The employee will be able to view important project deadlines on the left-hand side, with their weekly shift displayed on the right-hand side.

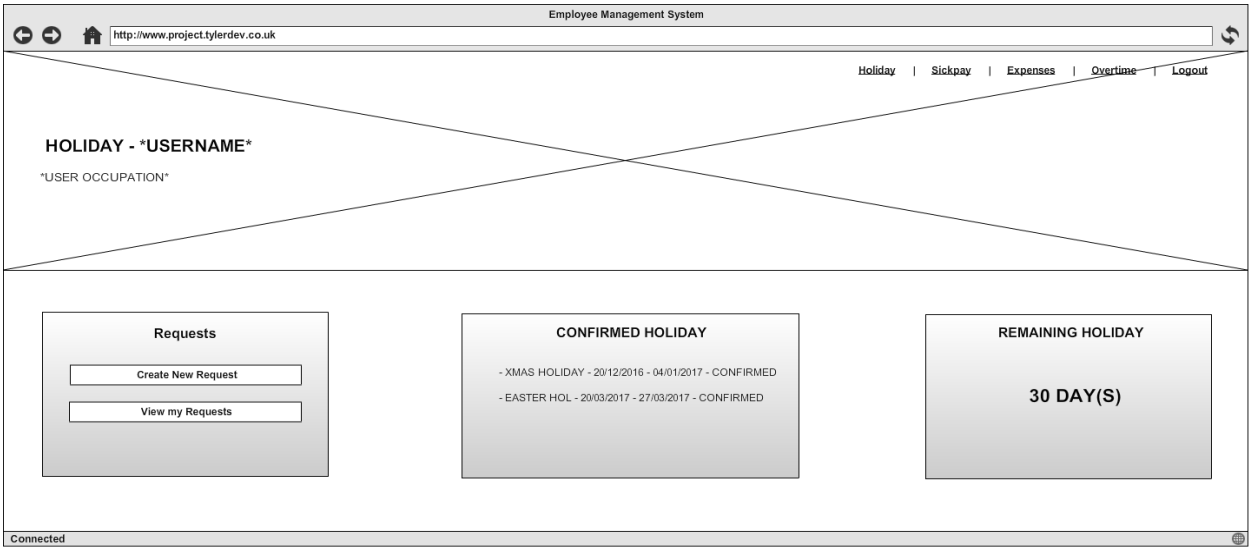
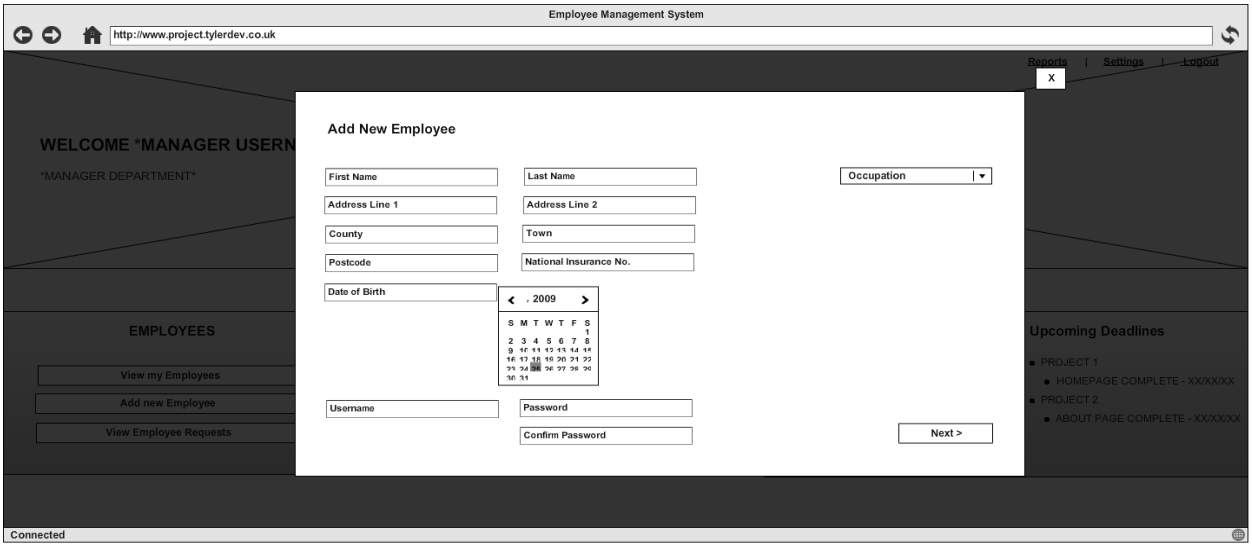


Figure 16 - Holiday Page Wireframe

To handle the holiday functionality, there will be a separate page containing the in-depth features. Following a similar layout to the dashboard, the holiday page will consist of the header with a relevant background image, and three blocks with the required features. They will be able to request a holiday, view all current requests, and their remaining holiday.



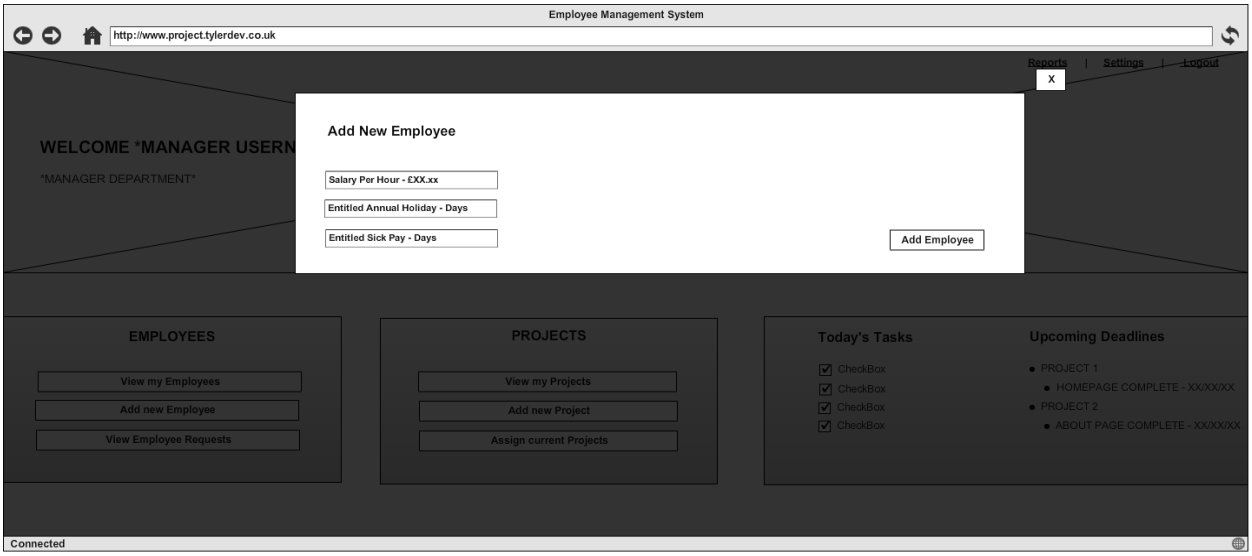


Figure 17 – Add New Employee Wireframe

The issue of redirecting to a separate page for each feature seemed implausible, and creates an issue regarding application navigation. As a solution, the input fields will be presented in the form of a modal. This is essentially a fully responsive popup screen across multiple devices in which the features are contained. Once the process is complete, the modal will close and direct the user to the page they were previously on.

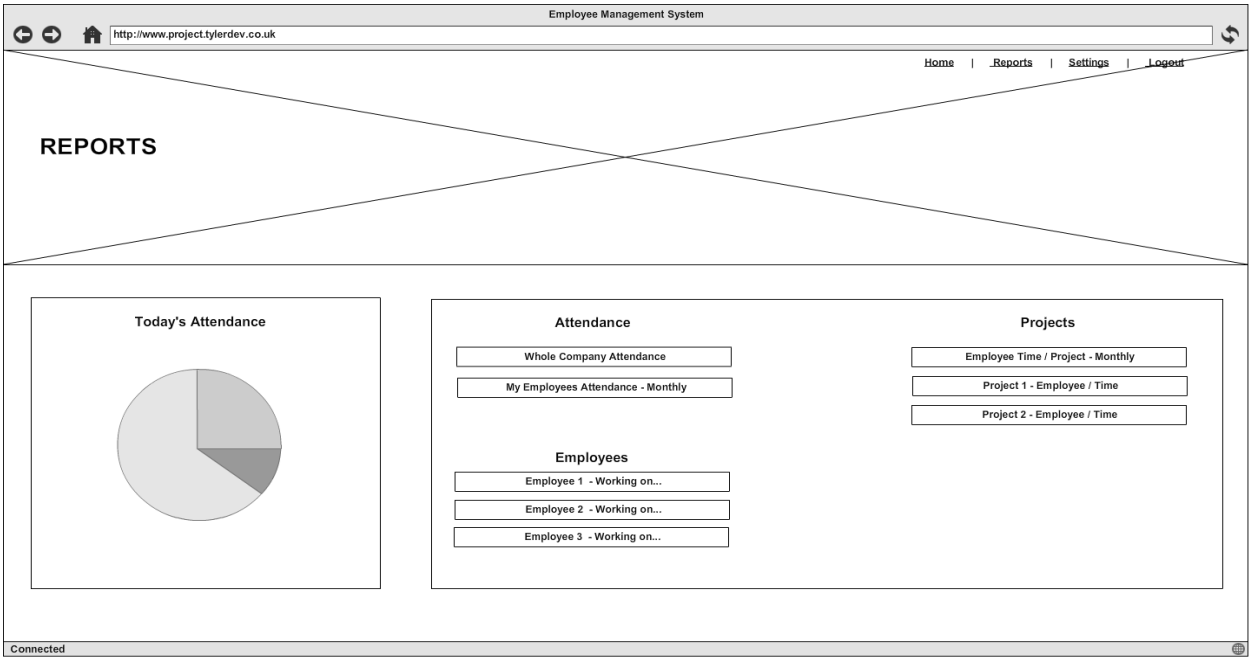


Figure 18 – Report Page Wireframe

The report page on the Manager profile will contain the ability to generate PDF formatted reports based on time, attendance, and various other project/employee related variants. A graph will be displayed on the left-side of the page constructed from daily statistics. The manager dashboard also

provides basic report viewing functionality, but due to the feature rich aspect of this functionality, an additional reports page was constructed.

4.3.2 - Pages

This application will consist of multiple HTML/PHP pages to accompany overall system functionality. Identifying these pages and the users that can access them, is one main aspect of a successful design plan. User privileges will be achieved with the use of PHP sessions, querying the database and jQuery hide/show functions.

Page	Employee	Manager	Administrator
Employee Dashboard	✓		
Holiday	✓		
Overtime	✓		
Manager Dashboard		✓	
Requests		✓	
Admin Dashboard			✓
Attendance Report		✓	✓
Settings	✓	✓	✓
Settings w/ view customisation			✓
Assign Manager/Employee			✓
Add Employees		✓	✓
Projects		✓	✓
Reports		✓	✓

Table 8 – Page Access by User Type

4.3.3 - Wireframe Discussion

A revision on the current wireframes has been established based on client feedback. Focusing on application usability, the height of the banner image located across most page templates was considered slightly too large to accompany the substantial number of features available. The solution to this problem was to keep the same layout, but simply reduce the height of the banner itself. This revision will be implemented during the development phase.

An additional revision to the original wireframes was the list of 'buttons' located within each segment on the dashboard. The client stated that the dashboard seemed cramped, atheistically. To combat this issue, these buttons will contain an 'overlay' with an icon and title, representing the functionality it contains. When hovering over these segments, the buttons will appear via an animation.

Excluding the amendments stated by the client above, the wireframes were considered a success and each influential stakeholder was satisfied with the representation of the finalized product.

4.3.4 - Menu Design

When researching into modern web applications, it became apparent that developing for the mobile platform is typically the first and most prioritised action of implementation. Considering the overwhelming statistics in favour of mobile browsing compared to the desktop medium means

catering for this audience is imperative. The client has also specified the requirement for the full operation of this web application on a mobile device. A major characteristic of mobile applications is general usability and fluid navigation. To ensure this feature is implemented, the design of menu items and the presentation of content was carefully considered.

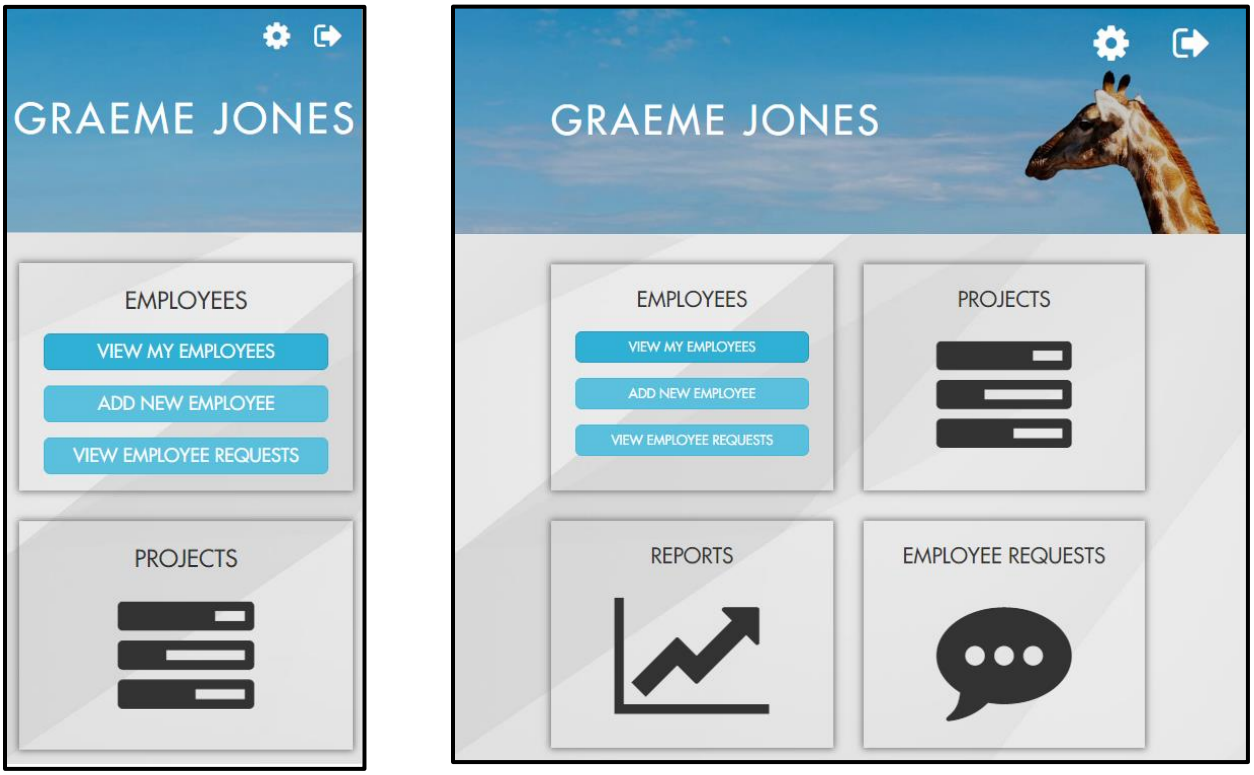
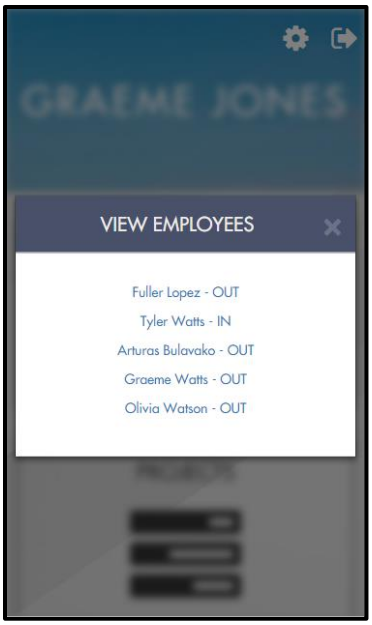


Figure 19 – Menu Items Mobile & Tablet

The above menu items display on hover when viewing from a desktop device, and on click when viewing from mobile. This is to ensure the usability principle of consistent functionality and views are applied across all viewports. The desktop site will display most information with the use of modals. This will also be the case with mobile devices. This was the defining reason for using modals, as the navigation on a mobile device is much more fluent without having to redirect to new pages.



As seen in the diagram on the left, the mobile design using a modal will allow the user to see their previous position on the page whilst opening new content. This feature is used to assure the user that their current position is still accessible, and invokes a sense of comfort, that no action up to that point has been dismissed. This design feature will be deployed across the entire application, resulting in a multi-layered, dynamic, up-to-date system

4.3.5 - Layered Structure

The concept of a layered structure within design views, is to reduce the need for the user to exit a webpage. It allows for the navigation of extensive content without relying on loading times and excessive page switching. It permits the user to complete tasks without losing their original position on the view. There are multiple ways of achieving this, but within this project it will be applied with the use of bootstrap modals and ajax (Asynchronous JavaScript and XML) functions.

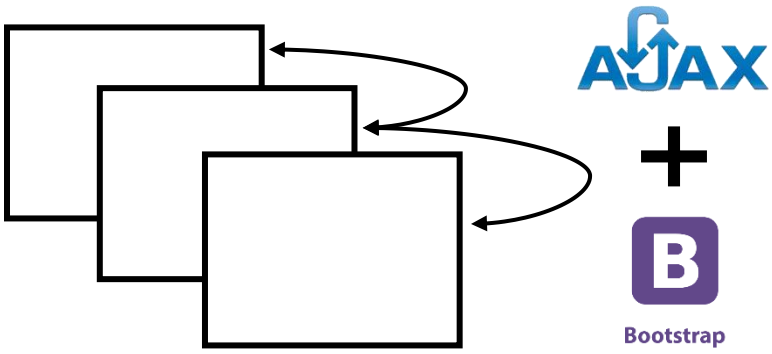
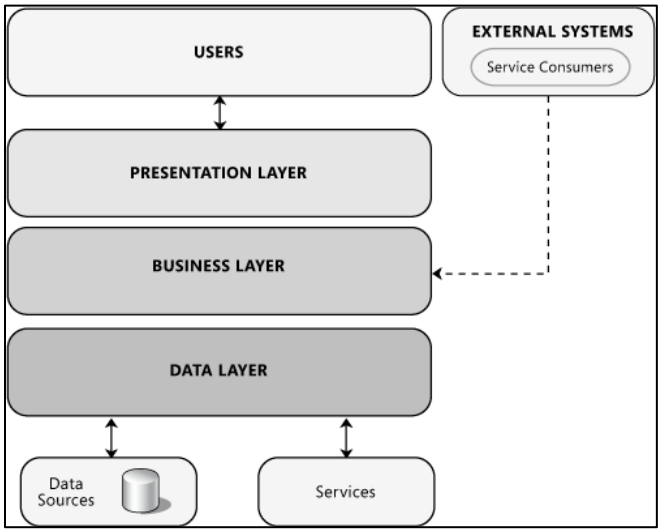


Figure 20 – Layered Structure Diagram

Layered design may also refer to the way the system components interact with one another. Visualizing these aspects during the design stage allows for efficient development and path clarity when implementation begins. Referring to these components as contained within different layers, aids with the design justification choices.



The presentation layers will relate to the HTML views, that will be present within the folder directory of the system. Presentation layer will be the use of third party libraries and API's such as jQuery. The data layer will essentially be the controller. This will be represented by php code and calls to the database. The data sources refers to the relational database stored on the webserver, which contains all relevant information being requested by the query. This simple layered design aspect has aided in implementation planning of important data communication.

Figure 21 – Layered Functionality Structure Diagram - ***(Msdn.microsoft.com, 2017)***

4.3.6 – Early Prototype

With the addition of wireframes, visual prototypes help with the CSS implementation, especially across multiple viewport widths. The following are some example prototypes, as presented and approved by the client, as to how some of the webpages will look.

The inclusion of the brand logo will be present across the entire application in the appropriate locations. This branding logo will be completely customizable as will the background images by the site administrator account.

Figure 22 – Login Prototype Mobile & Tablet

These prototypes were developed on a program called inVision designs. Creating these prototypes with comments allows the developer to imagine how the implementation is going to occur and document it well in advance.

4.4 - Conclusion

By covering the two main design phases; UX & Interfaces and Data Modelling, both client and developer gain an insight into how the interfaces are going to look, and validate the design choices with the database design (ERD), activity processes, and data dictionary. The finalized design decisions came as a result of carrying out the above tasks. By employing an iteration-based software development methodology, every segment of the design process has been revised numerous times and the application has developed greatly from the starting point.

Regarding Data Modelling, the Entity Relational Diagram has almost doubled in size and functionality since the preliminary meeting with the client. Conversing with the commissioner resulted in functionality clarity and forced a re-think to the entire relational design. This is a prime example of how iteration-based methodologies help with the progression and development of software applications.

5.0 - Implementation Chapter

During this chapter, the discussion of all implementation-related procedures will be detailed. Aspects such as tools and technologies, system architecture, database implementation, requirements implementation, and personal reflections on the entirety of the process.

5.1 - Tools & Technologies

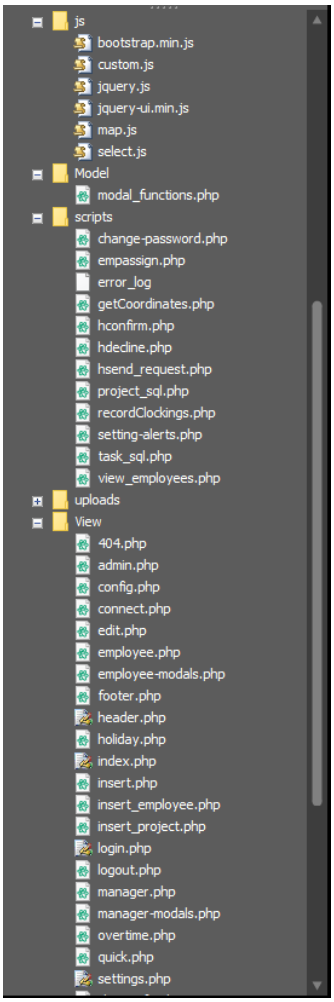
Numerous tools and technologies have been used in the design and implementation of this project. The choice of the following utensils was based around experience, competence, familiarity, and appropriateness.

Tool	Description	Use
Microsoft Expression Web 4	A smart multi-platform text editor for implementing code	Text Editor
Notepad++ w/NFTP	A simple text-editing software with an FTP plugin to transfer files directly to server	Text Editor
HTML 5	The mark-up language currently used to build the structure of content on the web.	Mark-up Language
CSS 3	The latest version of Cascading Style Sheets, currently used to style content on the web.	Style Sheet
PHP	A server-side scripting language used primarily for the functionality of web-based applications	Server-side Script
MySQL	An open source database management system, used to create and store data within databases	Relational Database Management System
Bootstrap	An open source framework with pre-defined classes and JavaScript functionality for fully responsive web pages.	HTML, CSS, and JS Framework
jQuery	A JavaScript library used specific functionality.	JavaScript Library
JavaScript	A client-side object-oriented language used for browser-specific functionality.	Client-side Script
php MyAdmin	An open source online software to manage the operating of MySQL database	Web Database Management System
StarUML	An open source modelling tool used to design and execute complex data modelling.	Unified Modelling Language

cPanel/WHM	A webserver for the hosting and setting up of the project domain	Hosted webserver

Table 9 – Tools & Technologies List

5.2 – Architecture



The word 'architecture' refers to the method in which something is constructed. In context to this application, it refers to the way in which the functionality and presentation has been structured. When researching into modern web applications and general software development principles, it became apparent that the most secure and efficient software architectural pattern was '***Model, View, Controller' (MVC).*** Having already developed a portion of the application using a different structural method, the finalised architecture concluded as an adaptation of MVC, adhering to the core principles. As seen in figure 23, the principle of separating concerns has been considered and employed across the file directories.

The files within the 'View' folder are the HTML pages consisting primarily of HTML code and minimal php (some conditional logic and echoing of variables). Many components within these pages will call upon files within the 'scripts' folder. This folder is the equivalent of the controller element in the MVC pattern. To resolve the issue of the 'view' being included within the URL, a redefinition of the root directory was altered in the .htaccess file to point the default root path to the view folder.

Like MVC, the 'scripts' (controller) folder contains only php, where much of the data processing occurs. Unlike MVC, the use of SQL queries is contained within the scripts files. This design decision was purely down to project management concerns, and time allocation could potentially be compromised when modifying the overall data processing unit. Aside from the slight deviation, this project is considered MVC.

Figure 23 – MVC file structure

The reason for the MVC pattern influences is that it possesses numerous benefits over other architectures. The separation of concerns allows for different sections of the project to be worked on without apprehensions about how the other sections function. The concentration remaining on one area at a time increases the overall quality of the application. This is especially the case as a single developer focussing on developing an entire application from scratch.

5.3 – Building the Graphical User Interface (GUI)

The Graphical user interface was built using a combination of HTML, CSS and JQuery. The wireframes within the design phase were presented to the clients and critiqued, revised, and ultimately approved. From the point of sanction, the implementation of the code began with the development of views. The primary views for both dashboards essentially follow the same layout, in that there is a banner stretched across the top, detailing the page contents and including the navigation menu items.

The defining of CSS and JavaScript was made in a php file named 'header.php'. This template part is included as standard across all views, resulting in the application of the styling and functionality remaining consistent throughout the system. The link to the CSS file, 'mobile.css', was the initial stylesheet to be created. To apply the width restrictions, a media query detailing 480 pixels and below only were added into the link as an attribute. Particular attention was paid to the interface usability regarding mobile and tabular devices, as the majority of users browsing the web do so on a handheld device rather than desktop.

5.4 – Implementing the Database

The database administration tool used within this project is an open source database management system called MySQL. When registering for a hosting package with cPanel capabilities, MySQL database typically comes as standard. MySQL is currently the second most used database management system on the web according to search engine crawls, and in turn is extremely secure and reliable.

The following tables were created using SQL create table queries.

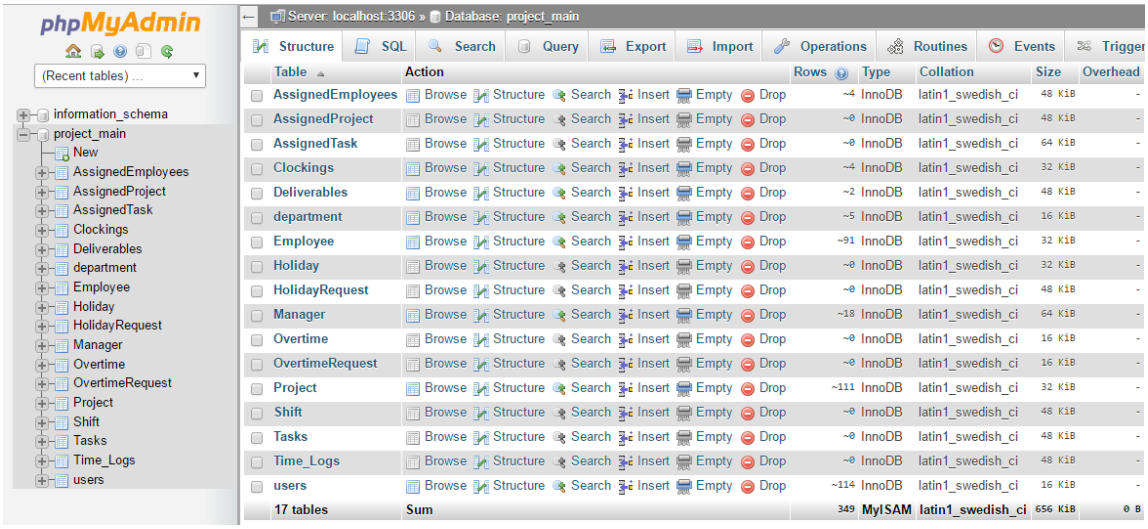


Figure 24 – Relational Database Structure

5.4.1 – Establishing connection to the database

The connection to the database was established through a php function called '***mysqli_connect***'. This creates a connection to the MySQL server by passing the database login details as parameters, and returns the value in the form of an object.

```php
<?php

    $connection = mysqli_connect('project.tylerdev.co.uk', 'project_admin', '[redacted]');
    if (!$connection){
        die("Database Connection Failed" . mysqli_error($connection));
    }
    $select_db = mysqli_select_db($connection, 'project_main');
    if (!$select_db){
        die("Database Selection Failed" . mysqli_error($connection));
    }
```

Figure 25 – Database Connection

The following code is contained within a php file named '***connect.php***'. This file is located within the Model folder of the public directory. For each page that performs a request/insert/update to the database, this file is referenced through a php '***require_once***' function. As this is required within most views, the file is referenced within the '***header.php***', which in turn is referenced on every view. This ensures a consistent and secure connection to the database throughout the entire application.

5.4.2 – Joining Tables

Joining tables is an extremely important aspect of relational database functionality. For a successful join within the database, the primary key of another table is included within the 'main' table via a foreign key. By doing this, records contained within another table can be referenced via this key. Foreign keys are considered a data-field constraint.

Column	Foreign key constraint (INNODB)		
Project_ID	`project_main`.`Project`.`Project_ID`	Constraint name	AssignedProject_ibfk_1
Manager_ID	`project_main`.`Manager`.`Manager_ID`	Constraint name	AssignedProject_ibfk_2

Figure 26 – Foreign Key Index

An example within this project's relational database would be the 'AssignedProject' table. This table contains the foreign keys (FK) for both the 'Project' and 'Manager' tables. These two foreign keys are referenced and together create a composite primary key. This means both of these records together must create a unique primary key for this table. By doing this, the manager can be assigned to a project, and this can be queried on the view (to show manager specific projects).

```php
$sql = "SELECT Project.Name, AssignedProject.Manager_ID
        FROM Project
        INNER JOIN AssignedProject ON ( Project.Project_ID = AssignedProject.Project_ID )
        INNER JOIN Manager ON ( AssignedProject.Manager_ID = Manager.Manager_ID)
        INNER JOIN users ON (Manager.User_ID = users.User_ID)
        WHERE Manager.User_ID = ".$user_id."";

$result = $connection->query($sql);
```

Figure 27 – Retrieve Assigned Projects SQL Query

The way these foreign keys are referenced within the view is with joins. INNER JOINS 'link' the two tables by defining the key of one table as that of another. By doing this, multiple attributes can be selected from different tables, but referencing the same record. The above SQL statement queries all the projects assigned to the current logged in manager through the use of SESSION variables and table joining.

5.4.3 – Retrieving Data

The initial retrieving of data from the database is accomplished with the use of the '***mysqli_query***' function passing the SQL and connection variables as the two parameters. The result of this query is then stored into a variable of its own, and in the format of an object. To cycle through this object and obtain the rows, the function 'mysqli_fetch_assoc(result) is used'. This is done with a while loop, and an array in which each column value gets entered into.

5.5 – Building System Functionality

5.5.1 – Login Functionality

The login functionality was created using a HTML form and conditional php to query the database. With HTML forms, there are multiple methods used to submit the data inputted into the fields. For this form, the '***POST'*** method was implemented. This is highly recommended when submitting sensitive information such as user passwords or bank details, as it sends the data to a definite source. The alternative method '***GET'***, is typically used to request data from a specified source, and appends these details within variables in the URL.

The action stated on this form is '***validate.php'***. This is the location that the request gets send to. From within this file, conditional php is used to determine the next processing action. As shown in figure 28 , the first conditional statement is determining if both the username and password fields have been filled out. If so, those details are stored into php variables. These variables are then used as conditions within a database query wherein it examines all records in the users table with the corresponding username and password. The php function '***mysqli_num_rows'*** then determines the number of records matching the input fields. If this number returns no records, the user is redirected to the login screen with the error message specifying that the login details are incorrect. If the record is equal to one, the user details are stored within the session, and redirects to '***index.php***' where additional conditional logic determines which dashboard to display.

5.5.1.1 – Password Encryption

Security and validation must also be carefully considered when handling an individual's private data. To ensure a high level of security within this application, password hashing and error handling have been utilised. The password variable defined by the input field is passed through a php function called '***crypt'.*** This employs an encryption by passing the password through as a parameter. For added security and to prevent SQL injections, the encrypted password is then run through another php hashing process, which applies further encryption with '***SHA265***', magic_quotes_gpc and striplashes.

At no point throughout the encryption process is the password decrypted. The string from the login input is passed through the same encryption process. This string is then entered into the database query to compare the two encrypted strings. If successful, the user is able to login, if not the user is

redirected to the login page. Encryption is essentially 'one-way' and is never actually decrypted. This logic, coupled with the double encryption, SQL escaping, and tag removing process, ensures users passwords are completely safe against malicious intensions.

```php
<?php
    session_start();
    require_once('connect.php');

    if (isset($_POST['username']) and isset($_POST['password']))
    {
        $username = $_POST['username'];
        $password = crypt($_POST['password'], '}xf(S>3L7U&!py+');
        $hash = hash('sha256', (get_magic_quotes_gpc() ? stripslashes($password) : $password));
        $query = "SELECT * FROM users WHERE username='{$username}' and password='{$hash}'";
        $result = mysqli_query($connection, $query) or die(mysqli_error($connection));
        $row = mysqli_fetch_assoc($result);
        $count = mysqli_num_rows($result);

        if ($count == 0)
        {
            header( 'Location: login?error=1' );
            exit();
        }
        elseif ($count == 1)
        {

            $_SESSION['username'] = $username;
            $_SESSION['first_name'] = $row['first_name'];
            $_SESSION['user_id'] = $row['User_ID'];
            $_SESSION['user_type'] = $row['user_type'];
            $_SESSION['last_name'] = $row['surname'];

            header('Location:/');
            exit();

        }
        else
        {
            die(header("location:login.php?"));
            exit();

        }

    }
```

Multiple layers of password encryption with php functions to prevent malicious SQL injections.

- Crypt()
- SHA265
- get_magic_quotes_gpc()
- stripslashes

Figure 28 – Login Validation Php w/ Password Encryption

5.5.2 – User Privileges

User privileges must be implemented correctly due to the vast diversity between both managers and employees' functionality access. This application will also hold and display sensitive user information, which means a potential breach of the Data Protection Act could be argued in the case of improperly considered user privilege conditions.

user_type
0
1

Within the user table, there is a column named 'user_type'. Depending on where the new user is being created within the application, will determine on the user type. For example, when a manager is creating a new employee, the user type will be entered as the value '0'. If an administrator was creating a new manager, the user type would be the value, '1'.

```php
<?php
    if($_SESSION['user_type'] != '1'){

        header('Location: /');

    }
?>
```

Figure 29 – PHP user type condition

From the login process, the values from the matching user record are stored into php session variables. Defining these session variables within a php required folder allows the access of this information across the entire application. This allows for php conditions to be created based on user type, and therefore hide/show specific information. On manager-access only views, the following if statement is applied at the top of each page;

The php header location function redirects to the url stated within the location string. In this case, the redirect will go to the domain address followed by '/' (index), and prevent any unauthorised access to those views. The same applies to template parts. Within the project view there is a required php call to another page labelled 'project-modals'. If the user were to try and access this page directly regardless of user type, the CSS, JavaScript and other important links will not be applied, and application integrity could be compromised. To combat this from happening, each template part contains the following code;

```
if(!defined('RESTRICTED')){
    exit('No direct script access allowed!');
}
```

→ No direct script access allowed!

Figure 30 – PHP restrict direct file access

5.5.3 – Attendance Tracking

The capability to be able to monitor and track attendance is a significant feature to be included within the application. Managers are required to be able to generate reports (visual graphs) of attendance records of individual employees, as well as their team as a whole. To accomplish this, the employee dashboard contains an attendance section, of which they are confronted with from the time of successful login.

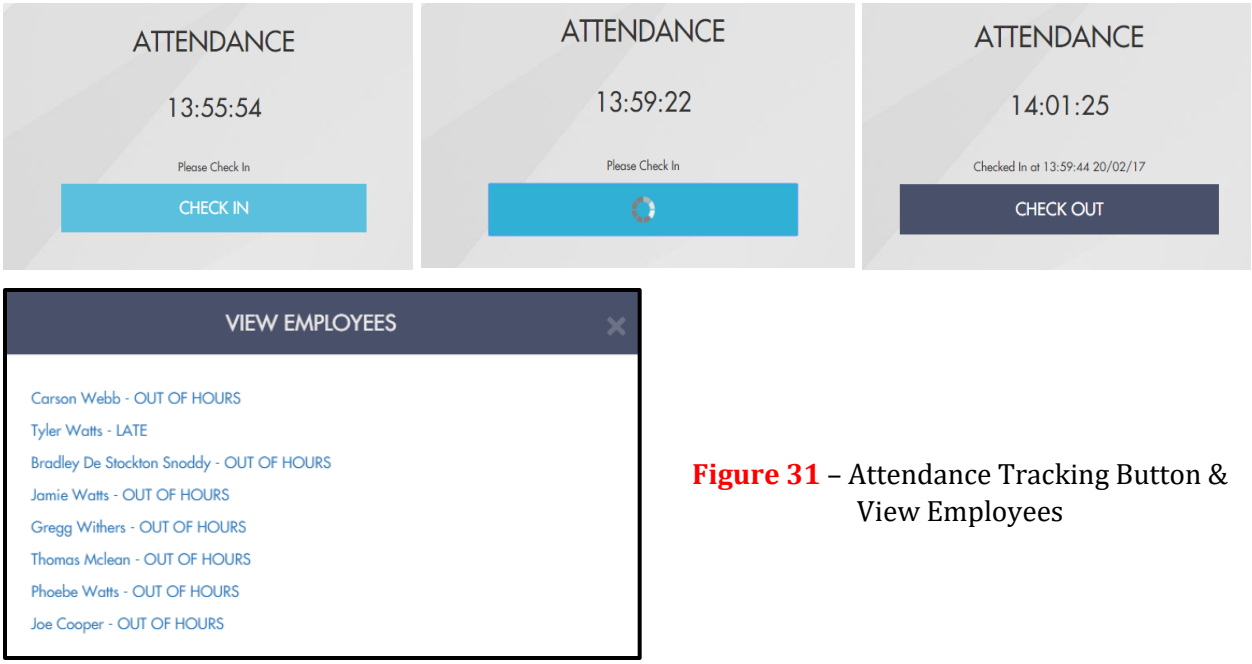


Figure 31 – Attendance Tracking Button & View Employees

As shown above, the employee will be met with a live clock and a button to check in or out (depending on their current status). Their status is determined by querying the database and seeing whether they're last attendance record has the clocking record 'in' or 'out'. This query also limits the search to the current day, one record, and is ordered by most recent. This way it ensures the attendance record is correct and up-to-date daily. This also allows the manager to view a live status list of all their employees.

On the topic of live attendance tracking, the reports section of the manager's dashboard provides a visual illustration of their current workforce's attendance. Below is a walk-through of how this was accomplished.

```
$sql =
    "SELECT users.User_ID, users.first_name, users.surname
    FROM users
    INNER JOIN Employee ON ( users.User_ID = Employee.User_ID )
    INNER JOIN AssignedEmployees ON ( Employee.Employee_ID = AssignedEmployees.Employee_ID)
    INNER JOIN Manager ON (AssignedEmployees.Manager_ID = Manager.Manager_ID)
    WHERE Manager.User_ID = ".$user_id."";

    $result = mysqli_query($connection, $sql) or die(mysqli_error($connection));
    $result_num = mysqli_num_rows($result);
```

Figure 32 – Get Assigned Employees Names SQL Query

The above SQL statement queries and retrieves the details of the employees under their assigned manager. When looping through the result of this query to obtain individual records, additional queries can be created based on each row's attributes.

```
while($row = mysqli_fetch_assoc($result)){

    $shiftSql = "SELECT * FROM Shift WHERE User_ID = '".$row['User_ID']."'";
    $shiftResult = mysqli_query($connection, $shiftSql) or die(mysqli_error($connection));
    $shiftRow = mysqli_fetch_row($shiftResult);

    $shiftStart = $shiftRow['2'];
    $shiftEnd = $shiftRow['3'];
```

Figure 33 - Get Assigned Employees Shifts SQL Query

Using the 'User ID' from the previous results, arguments can be constructed for the new query. The above query will get each user's shift start and end times, and set them into variables. This will allow for the check in/check out time of each user to be compared against their shifts.

```
$newSql =   "
                SELECT * FROM Clockings
                WHERE User_ID = '".$row['User_ID']."'
                AND dts >= CURDATE()
                AND In_Out = 1
                ORDER BY dts ASC
                LIMIT 1
            ";

$new_result = mysqli_query($connection, $newSql) or die(mysqli_error($connection));

while($new_row = mysqli_fetch_assoc($new_result)){

    $i++;
}
```

Figure 34 – Get Today's Check In Time

This query searches the 'Clockings' table for each user using the following arguments; today's date, checked in, order by most recent, and a limit of one. The while loop runs through and adds one to the variable each time those conditions are met (counts the number of records). From this point, the number of employees checked in can be compared against the overall number of employees.

```
$inHouse =  $i;
$outHouse = $result_num - $inHouse;
```

Figure 35 – Set Number of In and Out Employees in php Variables

The above variables contain the overall number of employees, and the sum that are currently checked in. Now the sums of the results are stored in php variables, they need to be passed into JavaScript variables to be used in conjunction with the jQuery addon, ChartJS.

```
var inHouse = " <?php echo $inHouse ?>";
var outHouse = " <?php echo $outHouse ?>";

   var dateJSArray = [
   {
       value: outHouse,
       color:"#F7464A",
       highlight: "#FF5A5E",
       label: "Absent"
   },
   {
       value: 0,
       color: "#FDB45C",
       highlight: "#FFC870",
       label: "Late"
   },
   {
       value: inHouse,
       color: "#46BFBD",
       highlight: "#5AD3D1",
       label: "On Time"
   }

];

var ctx = $("#canvas").get(0).getContext("2d");
var LineChartDemo = new Chart(ctx).Pie(dateJSArray, {

       segmentShowStroke : false,
       segmentStrokeColor : "#fff",
       segmentStrokeWidth : 2,
       percentageInnerCutout : 0,
       animationSteps : 90,
       animationEasing : "linear",
       animateRotate : true,
       animateScale : true
   }
);
```

The variables are converted by echoing out the php in JavaScript. Now these results are collated, they can be entered where necessary in the addon's API. Once the settings have been applied, the chart is built by appending the content to the ID, 'canvas'. Below is the result of a manager's employee's live attendance.

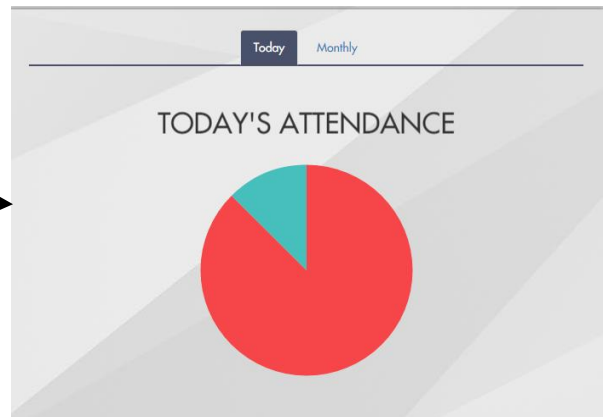


Figure 36 - Today's Attendance Pie Chart

5.5.4 – Setting/Tracking Tasks & Deliverables

The way project tracking is achieved is through the assigning of tasks in relation to that project. A high priority system requirement was for the manager to be able to set tasks based on a deliverable, and have that deliverable relate to a project. This was accomplished with the addition of a 'project' page, accessible only by the manager. A query is made to the database to retrieve the records of each project they have been assigned to. They will automatically be assigned to a project if they created it themselves. This query involves joining both the 'projects' and 'Assigned Projects' tables.

```
$manager = $_SESSION['manager_id'];

$sql = "SELECT Project.*, AssignedProject.Manager_ID
        FROM Project
        INNER JOIN AssignedProject ON (Project.Project_ID = AssignedProject.Project_ID)
        WHERE AssignedProject.Manager_ID = ".$manager."
        ORDER BY Project.Deadline ASC ;";
```

Figure 37 – Get all Assigned Projects SQL Query

The manager ID was stored within the session when successfully logging into the system, and is passed as an argument within the 'WHERE' clause to ensure the correct projects are being displayed. They're ordered by project deadline ascending, which makes sure the project with the closest deadline is always shown first.

Figure 38 – Project Page View

Once the projects are successfully displayed, the correlating deliverables also need to be requested. The above SQL statement is run within the previous while loop. This fetches the deliverables for each project via the where clause 'project ID'. These details are then loaded into an empty bootstrap modal via Ajax functionality. Ajax allows for call and response to/from the server without the page refreshing. By doing this, the multiple layered view architecture can be applied and important actions requiring processing by the server can be simultaneously carried out.

To call the tasks within each deliverable, the same SQL logic is applied. For each task, the modal will provide data from multiple tables including 'tasks, assigned tasks, deliverables, and projects'. Providing the task hasn't previously been assigned, the manager will be confronted with a select drop-down menu, in which they are able to choose an employee to carry out the task. Once submitted, the employee will be notified of this, and able to respond.

```
$newsql = "SELECT * FROM Deliverables
           INNER JOIN AssignedProject ON (Deliverables.Project_ID = AssignedProject.Project_ID)
           WHERE Deliverables.Project_ID = ".$row['Project_ID'].";";

$newresult = mysqli_query($connection, $newsql) or die(mysqli_error($connection));
$count = mysqli_num_rows($newresult);
```

Figure 39 – Get all Deliverables related to Project

The employee dashboard will contain a dropdown box of all current tasks. The employee will be able to click these tasks and obtain further details, similarly to the manager dashboard. Once the task has been chosen, a start button can be pressed. This inserts a start date into the record and informs the manager that the task is currently being undertaken. Once this task is complete, the employee will click the end button, resulting in an end time timestamp being updated into the database record. The manager will then be able to check whether the task has been completed to a satisfactory standard, and decide on the resulting actions.

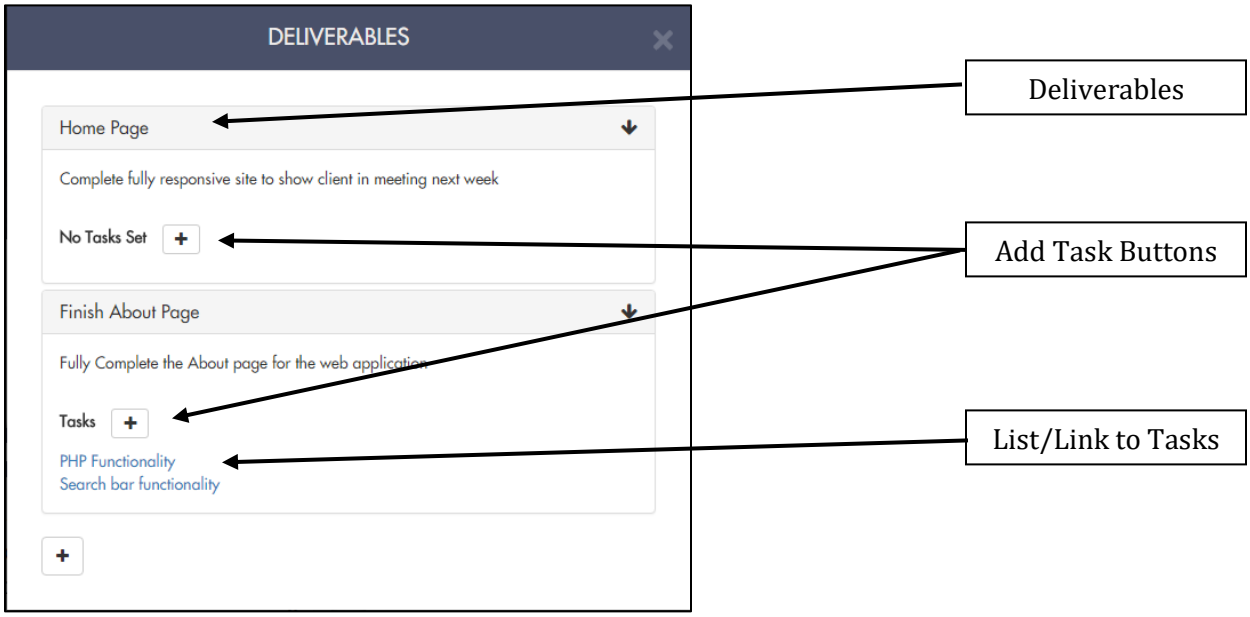


Figure 40 – Deliverables Modal

5.5.4 – Tracking time against Project

The calculation of time against a project can be accomplished through the relational aspect of the user/project/deliverable/task tables. As each of these tables are linked via a foreign key, the project ID can be deciphered from the task. Once a task is assigned to an employee, the employee ID can then be referenced to that project via the task assignment. The task assignment table contains both a start and end date field. Timestamp inputs are updated into these fields when an employee clicks either the begin or end of a task. From these two values, the amount of time can be calculated and entered into a separate table called 'Time Logs'. Once the 'Time Logs' records have correlated, queries to the database can include arguments within the 'WHERE' clause to retrieve them in order of the current day, week, month, or year.

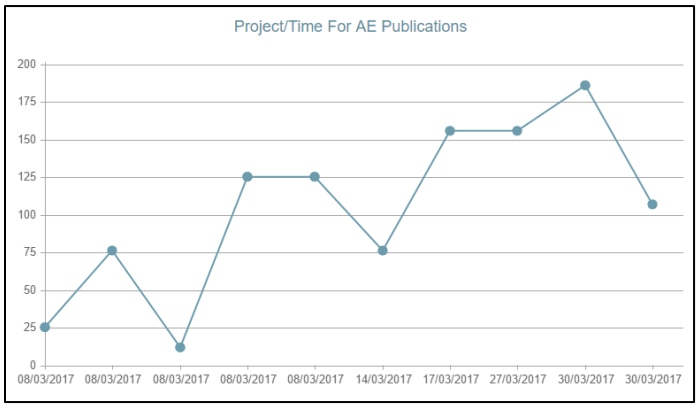


Figure 41 - Project/Time Graph

The values obtained from the 'time logs' can be stored into php variables and echoed into JavaScript. They can then be placed within the 'ChartJS' API settings to produce a line graph of minutes spent on the project. The diagram on the left displays the amount of time spent on a project on a daily basis. This will allow the manager to decide whether enough resource is being applied within the right places.

5.5.6 – Manager's Employees View w/ Google Maps API

A request from the client was for managers to be able to search and locate their employee's contact information quickly, should the occasion call for it. An example occasion would be if an employee was unexpectedly absent and had yet to inform their corresponding manager. This was implemented as an extension of the live attendance tracking system. As previously mentioned in '***5.5.3 – Attendance Tracking***', the manager has access to a list of their employees with their current attendance status. Building on top of this functionality, each of those employees became a link to a bootstrap modal, as seen below.

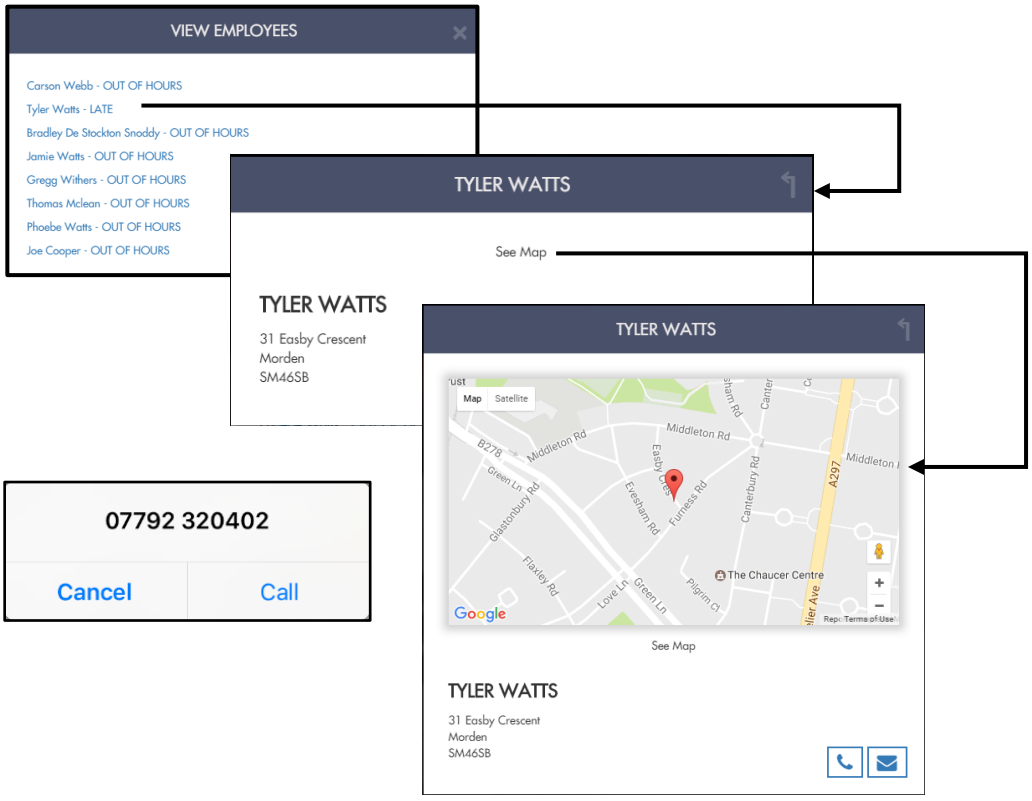


Figure 42 - Employee Modal W/Google Maps API

This bootstrap modal consists of the employee's contact information including address, integrated map, phone number and email address. The map is generated by using bespoke Google Maps API code, such as geolocation (to find employee address) and map options. Below are the steps to creating the automatic map.

```
$('[id^=my-employee]').on('shown.bs.modal', function (e) {

    if($('#map').length == '0') {
        $(this).find('.modal-body').prepend('<div id="map"></div>');
        var address = $(this).find('[id^=address]').attr('id');
        initMap(address);
        $(this).find('#map').delay(1000).hide();
    }

  });
```

This jQuery function declares that on modal open, look for the element with the id, 'Map'. If this does not exist, append that element. It then obtains the employee address from the records and passes the result through to the '***initMap***' function as a parameter.

```
var geocoder;
var map;

function initMap(address_id) {

        var map = new google.maps.Map(document.getElementById('map'), {
          zoom: 16,
          center: {lat: -34.397, lng: 150.644,
          streetViewControl: true
        }

    });

    var geocoder = new google.maps.Geocoder();
    geocodeAddress(geocoder, map, address_id);

}
```

Within the 'initMap' function, the map is created with multiple options such as zoom, center, and API functionality. To initialise the map with the employee's address automatically, an object known as a geocoder had to be created. Once the geocoder is created and stored within a variable, the address, map and geocoder are passed to the '***geocodeAddress***' function as parameters.

```
function geocodeAddress(geocoder, resultsMap, address_id) {

    var address = document.getElementById(address_id).value;
    geocoder.geocode({'address': address}, function(results, status) {
      if (status === 'OK')
      {
        resultsMap.setCenter(results[0].geometry.location);
        var marker = new google.maps.Marker(
        {
          map: resultsMap,
          position: results[0].geometry.location
        });
      }
      else
      {
        alert('Geocode was not successful for the following reason: ' + status);
      }

    });
}
```

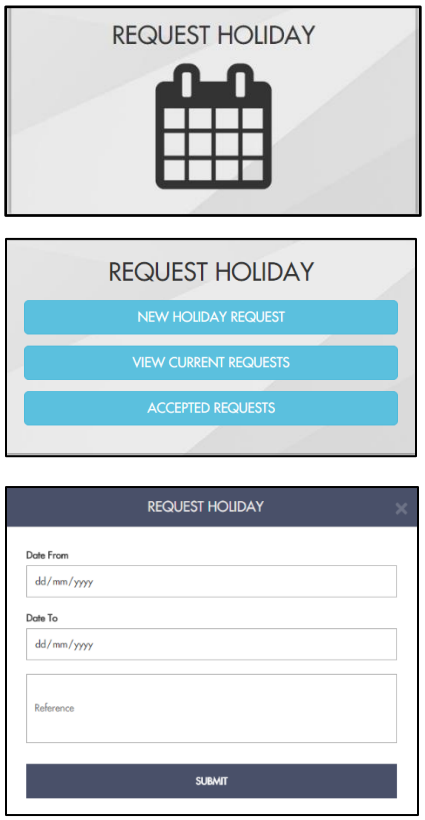
The '***geocodeAddress***' function receives the parameters from the previous function and applies them as arguments when defining the geolocate address and marker/map center. The if statement ensures that if the location cannot be generated, the user is alerted with a user message specifying the error, rather than just a blank map.

Figure 43 – jQuery & Google Maps API

This feature was also optimised for use on the mobile platform. The client stated that they may be away from the office regularly, and require the need to view employee's attendance status and contact them. The phone and envelope icons in the bottom right corner are both mobile compatible, and require one click to contact the employee through the selected method.

5.5.7 – Request Functionality

Each employee can send a holiday or an overtime request to their respective managers via the employee dashboard. Each type of request has its own separate page, with the ability to send new requests, view current requests, and view confirmed requests.




```
<div class="front-item">
    <h1>Request Holiday</h1>
    <i class="fa fa-calendar" aria-hidden="true"></i>

    <div class="item-content">
        <button type="button" class="btn btn-info btn-lg" data-toggle="modal" data-target="#holidayRequest">Ne
        <button type="button" class="btn btn-info btn-lg" data-toggle="modal" data-target="#currentRequest">Vi
        <button type="button" class="btn btn-info btn-lg" data-toggle="modal" data-target="#confirmedRequest">
    </div>
</div>

<div id="holidayRequest" class="modal fade" role="dialog">
  <div class="modal-dialog">

    <!-- Modal content-->
    <div class="modal-content">
      <div class="modal-header">
        <button type="button" class="close" data-dismiss="modal">&times;</button>
        <h4 class="modal-title">Request Holiday</h4>
      </div>
      <div class="modal-body">
        <div id="form-content">
            <form id="reg-form" method="post" action="/scripts/hsend_request.php" onsubmit="return

                    <label for="hol-start">Date From</label>
                    <input id="hol-start" name="start-date" type="date" placeholder="Date From">

                    <label for="hol-end">Date To</label>
                    <input id="hol-end" name="end-date" type="date" placeholder="Date To">

                    <input id="hol-ref" name="reference" type="textarea" placeholder="Reference">

                    <input type="submit" value="Submit">
            </form>
        </div>
      </div>
      <div class="modal-footer">
        <button type="button" class="btn btn-default" data-dismiss="modal">Close</button>
      </div>
    </div>

  </div>
</div>
```

Figure 44 – Holiday Request & PHP View

The above diagram shows the HTML code used to construct the employee request form and modal functionality. Once an employee completes a request, a record is entered into the request table and is viewable from the relevant manager's dashboard. The record entered contains a column named 'status'. To determine whether this request has been responded to, the default value is '0'. Once responded to by the manager, the status will either be altered to '1' or '2' depending on confirmation or declining.

Holiday Requests **Figure 45** – Employee Requests – Confirm & Decline

Name	Date From	Date To	Status	Confirm	Decline
Tyler Watts	22-04-2017	30-04-2017	Pending	✔	✖

When the manager logs into the system they are greeted with a dashboard, of which exists an employee requests section, implemented specifically to handle all employee requests. With the dropdown menu shown in this diagram, the manager can specify which requests they wish to view. This is created through an 'onchange' jQuery function, which sends a request to the server each time a select option is clicked. This allows for consistent up-to-date requests as the data is being fetched from the database each time.

```
if($q == '1'){

    $sql="SELECT users.first_name, users.surname, Employee.Employee_ID, HolidayRequest.*, Manager.Manager_ID, Manager.User_ID
    FROM HolidayRequest
    INNER JOIN Employee ON ( HolidayRequest.Employee_ID = Employee.Employee_ID )
    INNER JOIN Manager ON ( HolidayRequest.Manager_ID = Manager.Manager_ID )
    INNER JOIN users ON ( Employee.User_ID = users.User_ID )
    WHERE Manager.User_ID =".$u."
    ORDER BY Date_From DESC";

    $result = mysqli_query($connection,$sql);

    if($result){

        echo '<table>';
        echo    '<thead>';
        echo        '<tr>';
        echo            '<td>Name</td><td>Date From</td><td>Date To</td><td>Status</td><td>Confirm</td><td>Decline</td>';
        echo        '</tr>';
        echo    '</thead>';
```

Figure 46 – Append Request Table based on Select Option

5.6 – HTTPS & SSL

An application that stores and processes sensitive data must have the security protocols in place to ensure all measures possible have been applied to prevent any unauthorised access or manipulation. As is with this project, the storing and processing of user's personal data is a primary aspect to performing the desired functionality.

An SSL (Secure Sockets Layer) certificate was purchased and applied to the domain to ensure the link between the webserver and client browser is fully encrypted. Within the file structure, there is a hidden file named '*.htAccess*'. This file allows for some server modification and restrictions via bit code. ***(Db-engines.com, 2016)***

```
RewriteCond %{HTTPS} off
RewriteRule ^(.*)$ https://%{HTTP_HOST}%{REQUEST_URI} [L,R=301]
```

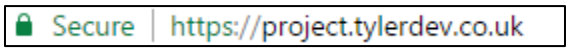


Figure 47 – HTTPS Redirect Condition - .htAccess

The code above redirects the user to 'https//-domain' whenever they access the site. By using HTTPS (Hypertext Transport Protocol Secure), the transmission of sensitive data across the network is safe-guarded with high level encryption. As well as the added security of having an installed SSL certificate and HTTPS protocol, the little symbol next to the domain within the corner

is a certified way of informing users that their data is safe. ***(Support.google.com, 2017***) It invokes a sense of business dependability, and increases client satisfaction in the form of a high quality functional application.

5.7 – Summary

Overall, the implementation of the project was a success. The functional requirements were met, and the iteration-based development was in-keeping with the principles of Agile methodologies. Having been fairly confident with the use of the tools and technologies, previous to the implementation, I now realise a great deal of knowledge was missing in terms of depth.

Despite the success in meeting the functional aspects of the application, certain components of the software could have been achieved in a more efficient way. The use of Ajax was introduced mid-development and was gradually learned along the way. If time allocation were not such a strict constraint, Ajax may have been applied elsewhere.

However, the integration of third-party jQuery libraries and API's made for interesting and challenging development. Using new technologies and php logic to intertwine code to achieve dynamic functionality made for an interesting and enjoyable experience, as well as a fully functional web application.

6.0 - Testing Chapter

During the course of this chapter, I will be testing the application to ensure all required functionality operates correctly and produces the desired results. Rigorous testing throughout and post development is vital to discovering any errors and resolving them before the final hand-over. There are numerous methods available to achieve this, and the ones applied are discussed below;

6.1 – Unit Testing

Unit testing is a form of software testing that involves the inspection of components within a system only. These components are individual pieces of code, that typically produce a single output. In context to procedural programming, unit testing would involve the checking of small amounts of code such as functions, procedures, or external plugin references. The method used to carry out unit testing is referred to as 'White Box Testing' (WBT).

6.1.1 – White Box Testing

WBT involves testing with the knowledge of the internal structure of a system. Typical test cases involving WBT are input validations and valid/invalid arguments to determine the reaction of the system. An advantage of this testing method is that the testing can typically take place earlier in the development life cycle by the developer.

6.2 – Functional Testing

Functional testing is the type of testing done relating specifically to the system's functional requirements. This is typically achieved through the completion of test cases. Without having a technical knowledge of the system's inner workings, functional testing focusses on ensuring the application outputs the correct result. Regarding this project, functional testing will be applied to ensure the correct results are displayed each time***. (Software Testing Class, softwaretestingclass.com, 2015)***

6.3 – Non-Functional Testing

Non-functional testing is the type of testing done relating to the non-functional requirements derived from the requirements analysis section. This is not so much the direct testing of the system, but more the 'readiness' of the application. - ***(Software Testing Class, softwaretestingclass.com, 2015)***

6.4 – Test Cases

Test cases are a method of performing functional testing. They're a record of tests carried out with the expected result, actual result, and final status. By carrying out test cases, it ensures vital inspection has been undertaken regarding the system's functional requirements. They help discover the existence of underlying errors, which may have gone un-noticed had they not been taken into consideration.

Login Test Case			
ID:	TC1	**Designed By**	Tyler Watts
Priority:	High	**Executed By:**	Tyler Watts
Name:	Login Functionality	**Execution Date:**	04/01/2017

Pre-conditions:
No sessions present on login page.
The user is not logged in.

Dependencies:
N/A

#	Test	Expected Result	Result	Status
1	Type in valid username and password	The correct user dashboard is displayed and session information set	PASS	Complete
2	Type invalid username and password	No sessions created, redirect to /login with error message	PASS	Complete
3	Try to access '/login' when already logged in	No access to '/login' when already logged in to prevent any session overlaps and access to unauthorized information.	FAIL	Please see '#4' for re-test.
4	RETEST of #3	No access to '/login' when already logged in to prevent any session overlaps and access to unauthorized information.	PASS	Complete
5	Login with manager username and password	Redirect to Manager Dashboard with correct user details set	PASS	Complete
6	Login with employee username and password	Redirect to Employee Dashboard with correct user details set	PASS	Complete
7	Logout from any page	Click logout button, sessions are cleared and user redirected to '/login'.	PASS	Complete

Notes:
#3 – 'If statement' defined in php to prevent access to '/login' if a user session is set.

Table 10 – Login Test Case

Employee Request Test Case			
ID:	TC2	**Designed By**	Tyler Watts
Priority:	High	**Executed By:**	Tyler Watts
Name:	Employee Request	**Execution Date:**	12/01/2017

Pre-conditions:
User logged in as an employee.

Dependencies:
Manager is assigned to Employee

#	Test	Expected Result	Result	Status
1	Employee clicks on holiday icon in navigation bar	The page redirects to the ***'holiday.php'*** view	PASS	Complete
2	Employee clicks on 'New holiday request'	A bootstrap modal opens with a form detailing the fields, 'date from', 'date to', and 'reference'.	PASS	Complete
3	The Employee submits the form with valid fields	An insurance message appears alerting the user to confirm they intend to send. The page redirects to '/holiday'.	PASS	Complete
4	The Employee submits the form with 'date from' field before current day	An error message appears alerting the user that a day from the past is an invalid value	FAIL	Please see '#5' for re-test
5	RETEST of #4	An error message appears alerting the user that a day from the past is an invalid value	PASS	Complete
6	The Employee submits a range of days larger than remaining holiday	The user is presented with an error message detailing the problem. Form is not submitted	PASS	Complete

Notes:
#3 – Form validation was applied for the fields within the request form
Although this test case is relating to holiday, the other requests follow the same method and have undergone the same testing procedure.

Table 11 – Employee Request Test Case

Manager View Employees Test Case

ID:	TC3	**Designed By**	Tyler Watts
Priority:	High	**Executed By:**	Tyler Watts
Name:	Employee Request	**Execution Date:**	27/01/2017

Pre-conditions:
User logged in as a manager.

Dependencies:
Manager has at least 1 assigned Employee.

#	Test	Expected Result	Result	Status
1	Manager clicks on 'View Employees' button on dashboard	Bootstrap Modal opens with a list of employees	PASS	Complete
2	Manager clicks on individual employee for further details	Additional Bootstrap Modal opens with employee details	PASS	Complete
3	Manager clicks on see map	Map slides down with address automatically searched	FAIL	Please see '#4' for re-test
4	RETEST of #3	Map slides down with address automatically searched	PASS	Complete
5	Manager clicks on 'View Employees' button on dashboard	List of employees has live status of current attendance in relation to their shifts	PASS	Complete
6	Phone Button	When clicked, the option to call with the employee's number is automatically initiated.	PASS	Complete
7	Email Button	When clicked, the option to email with the employee's email address is automatically initiated.	PASS	Complete

Notes:
#3 – Map was not initiating due to the jQuery sliding animation conflict. A delay was added to allow the map to initialize before hiding.

Table 12 – View Employees Test Case

Add Project/Deliverable/Task			
ID:	TC4	**Designed By**	Tyler Watts
Priority:	High	**Executed By:**	Tyler Watts
Name:	Employee Request	**Execution Date:**	14/02/2017

Pre-conditions:
User logged in as a manager.

Dependencies:
Manager has at least 1 assigned Employee.

#	Test	Expected Result	Result	Status
1	Add New Project	Bootstrap Modal opens with input fields for a new project	PASS	Complete
2	Submit Project Form	A new project is created without page reload using Ajax functionality.	PASS	Complete
3	Add New Deliverable	Bootstrap Modal opens with input fields for a new deliverable	PASS	Complete
4	Submit Deliverable	A new deliverable is created with selected project	PASS	Complete
5	Add New Task	Bootstrap Modal opens with input fields for a new task	PASS	Complete
6	Submit Task	A new task is created relating to the selected deliverable.	PASS	Complete
7	Invalid fields within forms	Form not submitted. Error message appears	PASS	Complete

Notes:
N/A

Table 13 – Add Project/Deliverable/Task Test Case

Employee/Manager Attendance Tracking			
ID:	TC4	**Designed By**	Tyler Watts
Priority:	High	**Executed By:**	Tyler Watts
Name:	Employee Request	**Execution Date:**	14/02/2017

Pre-conditions:
User logged in as a manager.

Dependencies:
Manager has at least 1 assigned Employee.

#	Test	Expected Result	Result	Status
1	Employee Clicks Check in button	The button loads and confirms the check-in with time and attendance status	PASS	Complete
2	Employee Clicks Check out	The button loads and confirms check-out with time and attendance status	PASS	Complete
3	Manager Clicks Reports Page	The page loads with report options and a pie chart of employee's live attendance	PASS	Complete
4	Manager Clicks 'View my employees button'	A bootstrap modal opens with a list of all assigned employees and their current attendance status	PASS	Complete
5	Manager clicks employee attendance report button	A bootstrap modal opens with last 30 days attendance report displaying line-graph	FAIL	Re-test Required
6	RETEST OF #5	A bootstrap modal opens with last 30 days attendance report displaying line-graph	PASS	Complete

Notes:
#5 – Error within the API scope resulted in graph not loading correctly. Now fixed.

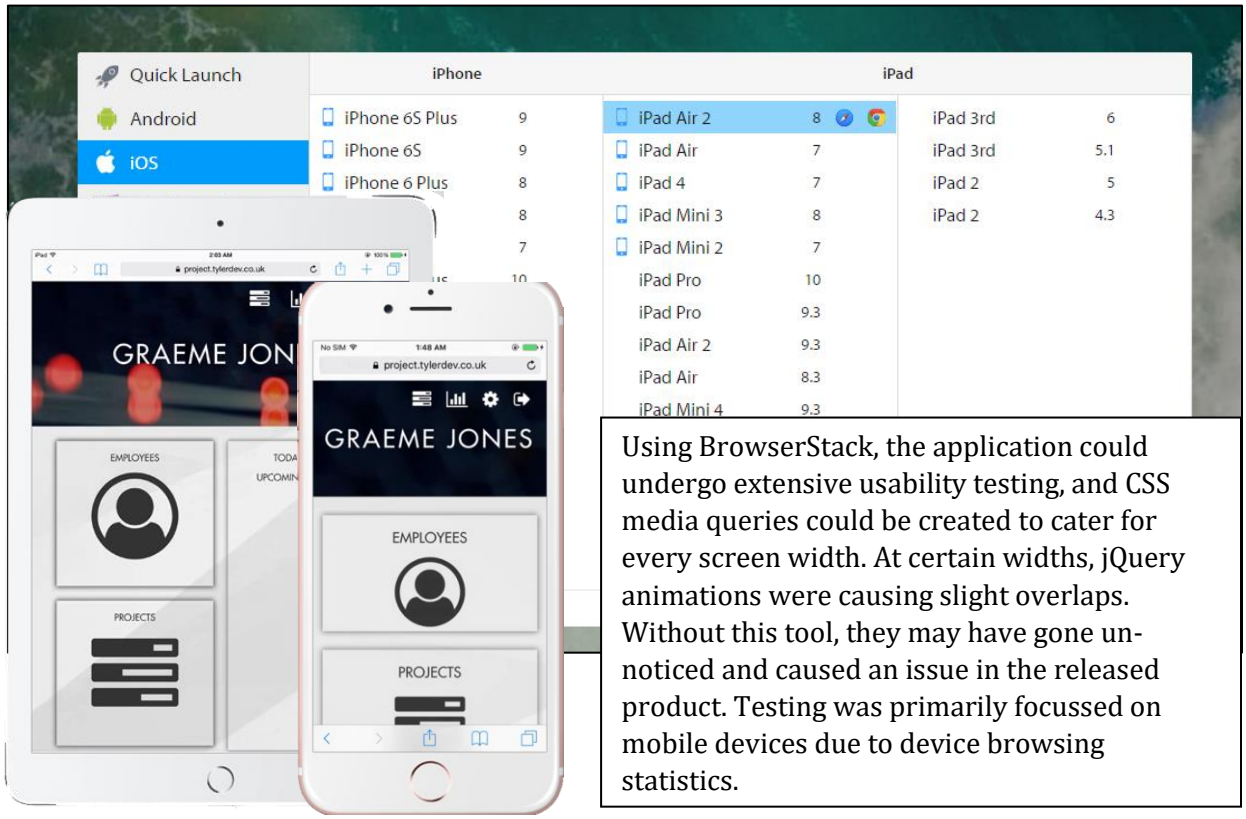
Table 14 – Employee/Manager Attendance Tracking - Test Case

6.5 – Usability Testing

Usability testing is the testing of the overall user experience when accessing and browsing the web application. Distinct browsers render CSS differently, and it can result in the user experience being fragmented and disjointed when viewing on various devices. The user must have a pleasant and consistent experience across multiple viewport widths, browsers, and operating systems. To ensure this project adheres to these usability principles as strictly as possible, the application has been executed through live device simulators, running on varied operating systems and browsers. The online resource to perform this task is called, 'BrowserStack'. Having purchased a premium account with this company, the application is able to be tested extensively.

The application has been tested on the following: ***– (BrowserStack, browserstack.com, 2017)***

Device	Browser	Operating System	Compatible	Notes
iPhone 5 & 5S	Safari	iOS 6	✓	
iPhone 6 & 6S	Safari	iOS 8	✓	
iPhone 7 & 7S	Safari	iOS 10	✓	
iPad Air 1 & 2	Safari	iOS 8	✓	
iPad Mini	Safari	iOS	✓	
MacBook Pro 13"	Safari & Chrome	Sierra	✓	Slight overlap issue with menu fixed
MacBook Pro 15"	Safari & Chrome	El Capitan	✓	
Lenovo Laptop 15"	Chrome & IE10	Windows 10 Windows 8.1	✓	



Using BrowserStack, the application could undergo extensive usability testing, and CSS media queries could be created to cater for every screen width. At certain widths, jQuery animations were causing slight overlaps. Without this tool, they may have gone un-noticed and caused an issue in the released product. Testing was primarily focussed on mobile devices due to device browsing statistics.

7.0 - Critical Evaluation

This chapter presents the prospect for personal reflection on the planning and implementation of the project as well as a review of the outcome. The following contains a detailed account of the challenges faced during various stages of the project life cycle, personal achievements, technical shortcomings, and an all-encompassing conclusion.

7.1 – Challenges Faced

There were many challenging aspects of designing and building distinct application characteristics. This was personally the most complicated project undertaken by myself to date. Below are challenges I faced within different phases of the development life cycle;

7.1.1 – Design Challenges

The designing of a system as complex as this employee/project management system was a demanding task, and was not without its challenges. The most thought-provoking aspect of the design was determining the relational integrities between the database tables. As an iteration-based development methodology was applied, there were continuous changes to the database designs, which sometimes overlapped with implementation alterations. A meeting with the client towards the latter stages of the project revealed the need for tasks to be assigned to employees. This change provoked an entire rethink regarding the measurement for time spent on each project. To achieve this, the link between project, deliverable and task had to be created using foreign keys. Also, to relate the task to employees, the addition of an assigned task table. With the use of start and end times, the time could be calculated and assigned to the relevant project.

7.1.2 – Implementation Challenges

A continuous backend error encountered during development was an SQL syntax error. This occurred on multiple occasions, and caused a somewhat lengthy delay with the progress of some developments. Having these files required within the view, the assumption was that the session start would apply to the entire page and therefore, the session variables would be valid for insertion into the query. After researching into scope, it became apparent that the php file was not in-fact reaching the session defining document, due to Ajax running the database query separately and returning the results to the view. This was realised by running the query results through a php ***'var_dump'*** function. When the result displayed null, the clarity regarding the syntax error became plainer. The variables containing the session details were not getting set properly, due to the file scope structure. Once defining these details within the php file itself, the variables became valid and the functionality began working.

7.2 – Project Insights

Whilst developing this application, the importance of the aspects involved within project development became apparent. The need for effective time management was crucial to the success of this application, and at times, was not considered carefully enough. Having experienced the pressure of upcoming deliverables, I personally feel my time management skills have greatly benefited.

7.3 – Critical Technical Review

The overall construction of this application was fairly fluent and comfortable. The choice of programming language, development IDE, design decisions, tools and techniques were fully justified and complimented each other well to create a requirement fulfilling web application. Having researched into each layer when developing, it became evident, there are further modern

and secure ways of achieving a similar result. The main method of communicating with the database within this application is with the use of 'mysqli'. A way through the project life cycle, the method of using PDO to communicate with the database became known to myself. The advantages of PDO over mysqli is that PDO is compatible with 12+ different database drivers, which allows flexibility and changes within data source (future proofing). PDO also allows for named parameters and prepare statements, which further removes the chances of malicious SQL injections.

Having reviewed these two methods, it is clear PDO is the more secure and modern method of connecting the application to the database. Had this knowledge been acquired pre-development, it would have most definitely been the primary method used.

7.4 – Future Functionality

Throughout development, the opportunity for additional functionality was always present. The client stated the possibility of incorporating a financial-management based module into the system to make it become a fully-functioning work 'hub' for all departments. However, due to time constraints and requirement prioritisation, this feature was unable to be implemented. If these development constraints were to be extended or lightened, the inclusion of such a feature would be viable and easily transitional due to the code reuse of current features already contained.

Without the financial management aspect of the application being implemented, the sick pay functionality became redundant. This functionality was removed during development, as it would have incorporated the salary accumulation functionality for the finance user to observe. The client was made aware of this situation, and agreed that it may later be introduced during an advanced version.

As one of the main aims of this application was to make the management of project-based tasks more efficient between manager and employee, the possibility of a messaging system was initiated. This type of functionality would increase the efficiency of communication between colleagues and contribute towards the overall system objectives, however, development related restrictions again halted this, and the feature was not considered of a significant enough priority. A substantial amount of research was carried out about how to employ this functionality, and the use of a third-party API called 'LiveChat' seemed the most feasible option (***LiveChat, 2017***). The system structure does cater for the addition of functionality, and if the opportunity arises to include this within a future version update, I'd personally be very confident of successfully applying it.

7.5 – Conclusion

The overall experience of having constructed this application has been extremely beneficial. Having undergone all the tasks required within software engineering, such as deriving functional requirements, creating dynamic designs, implementing functional software, and conducting extensive testing, has vastly developed my skill set as an aspiring professional within the industry. The independent problem solving involved across the entirety of this project has resulted in myself becoming self-sufficient and responsible for large portions of work across multiple sectors.

References & Bibliography

1. Support.google.com. (2017). Secure your site with HTTPS - Search Console Help. [online] Available at: https://support.google.com/webmasters/answer/6073543?hl=en [Accessed 6 Feb. 2017].

2. Cps.gov.uk. (2017). Data Protection: Legal Guidance: The Crown Prosecution Service. [online] Available at: http://www.cps.gov.uk/legal/d_to_g/data_protection/ [Accessed 10 Jan. 2017].

3. Ico.org.uk. (2017). London NHS trust fined for HIV newsletter data breach. [online] Available at: https://ico.org.uk/about-the-ico/news-and-events/news-and-blogs/2016/05/london-nhs-trust-fined-for-hiv-newsletter-data-breach/ [Accessed 8 Feb. 2017].

4. Ico.org.uk. (2017). London council data breach as court bundle lost. [online] Available at: https://ico.org.uk/about-the-ico/news-and-events/news-and-blogs/2016/11/london-council-data-breach-as-court-bundle-lost/ [Accessed 8 Feb. 2017].

5. Ico.org.uk. (2017). London council data breach as court bundle lost. [online] Available at: https://ico.org.uk/about-the-ico/news-and-events/news-and-blogs/2016/11/london-council-data-breach-as-court-bundle-lost/ [Accessed 8 Feb. 2017].

6. Dataprotection.ie. (2017). Offences and Penalties under the Data Protection Act - Data Protection Commissioner - Ireland. [online] Available at: https://www.dataprotection.ie/docs/Offences_and_Penalties_under_the_Data_Protection_Act/97.htm [Accessed 9 Feb. 2017].

7. Data Protection Act 1998 Undertaking London Borough of Ealing. (2016). 1st ed. [ebook] London: Information Commissioner's Office. Available at: https://ico.org.uk/media/action-weve-taken/undertakings/1625364/london-borough-of-ealing-undertaking.pdf [Accessed 14 Feb. 2017].

8. Legislation.gov.uk. (2017). Data Protection Act 1998. [online] Available at: http://www.legislation.gov.uk/ukpga/1998/29/section/55 [Accessed 9 Feb. 2017].

9. Cole, C., McCullough, K. and Powell, L. (n.d.). Regulation of Reinsurance Recoverables: Protection or Protectionism?. SSRN Electronic Journal, 33(6).

10. Ico.org.uk. (2017). Exemptions. [online] Available at: https://ico.org.uk/for-organisations/guide-to-data-protection/exemptions/ [Accessed 12 Feb. 2017].

11. Data Protection Regulatory Action Policy. (2017). 1st ed. [ebook] Information Commissioner's Office. Available at: https://ico.org.uk/media/about-the-ico/policies-and-procedures/1853/data-protection-regulatory-action-policy.pdf [Accessed 10 Feb. 2017].

12. Itgovernance.co.uk. (2017). Data Protection Act (DPA) and EU GDPR Penalties. [online] Available at: https://www.itgovernance.co.uk/dpa-penalties [Accessed 11 Feb. 2017].

13. Bonner, L. (2012). Cyber Risk: How the 2011 Sony Data Breach and the need for Cyber Disk Insurance Policies Should Direct the Federal Response to Rising Data Breaches.

14. Info.ssl.com. (2017). What is SSL?. [online] Available at: http://info.ssl.com/article.aspx?id=10241 [Accessed 15 Mar. 2017].

15. Legislation.gov.uk. (2017). Data Protection Act 1998. [online] Available at: http://www.legislation.gov.uk/ukpga/1998/29/section/29 [Accessed 23 Apr. 2017]

16. Db-engines.com. (2016). DB-Engines Ranking - popularity ranking of database management systems. [online] Available at: https://db-engines.com/en/ranking [Accessed 22 Apr. 2017]. - (Db-engines.com, 2016)

17. Smith, C. and Siegal, J. (2016). Mobile internet usage surpasses desktop usage for the first time in history. [online] BGR. Available at: http://bgr.com/2016/11/02/internet-usage-desktop-vs-mobile/ [Accessed 14 Mar. 2017].

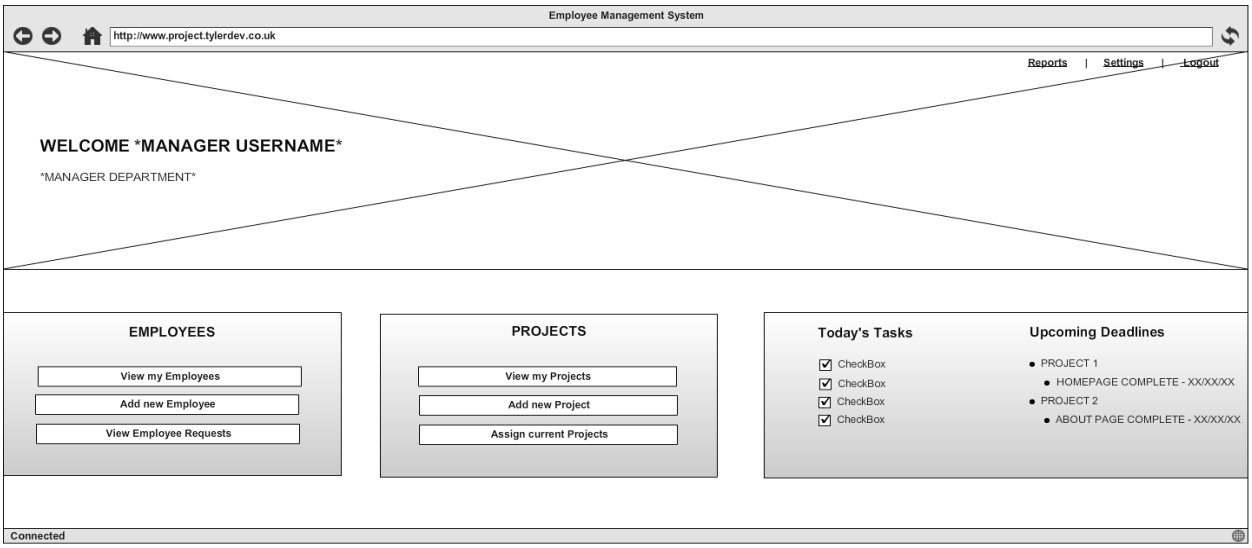
18. Software Testing Class. (2013). Functional Testing Vs Non-Functional Testing. [online] Available at: http://www.softwaretestingclass.com/functional-testing-vs-non-functional-testing/ [Accessed 7 Apr. 2017].

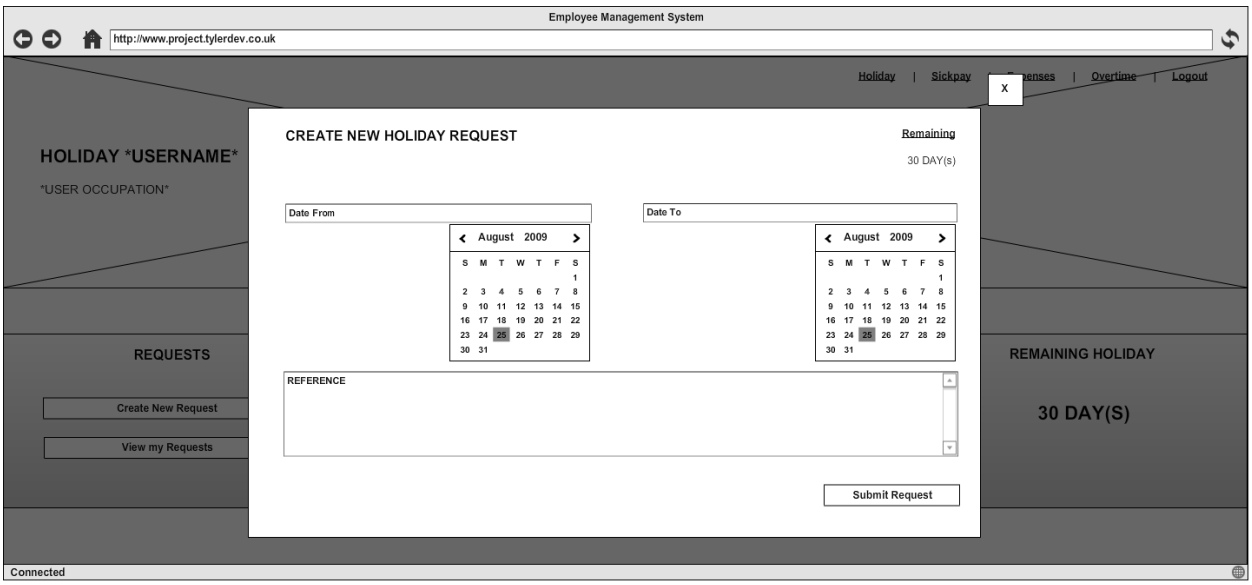
19. LiveChat. (2017). PHP examples. [online] Available at: https://www.livechatinc.com/api/v1/php-examples/ [Accessed 2 Nov. 2016].

20. Msdn.microsoft.com. (2017). Chapter 5: Layered Application Guidelines. [online] Available at: https://msdn.microsoft.com/en-gb/library/ee658109.aspx [Accessed 12 Mar. 2017].

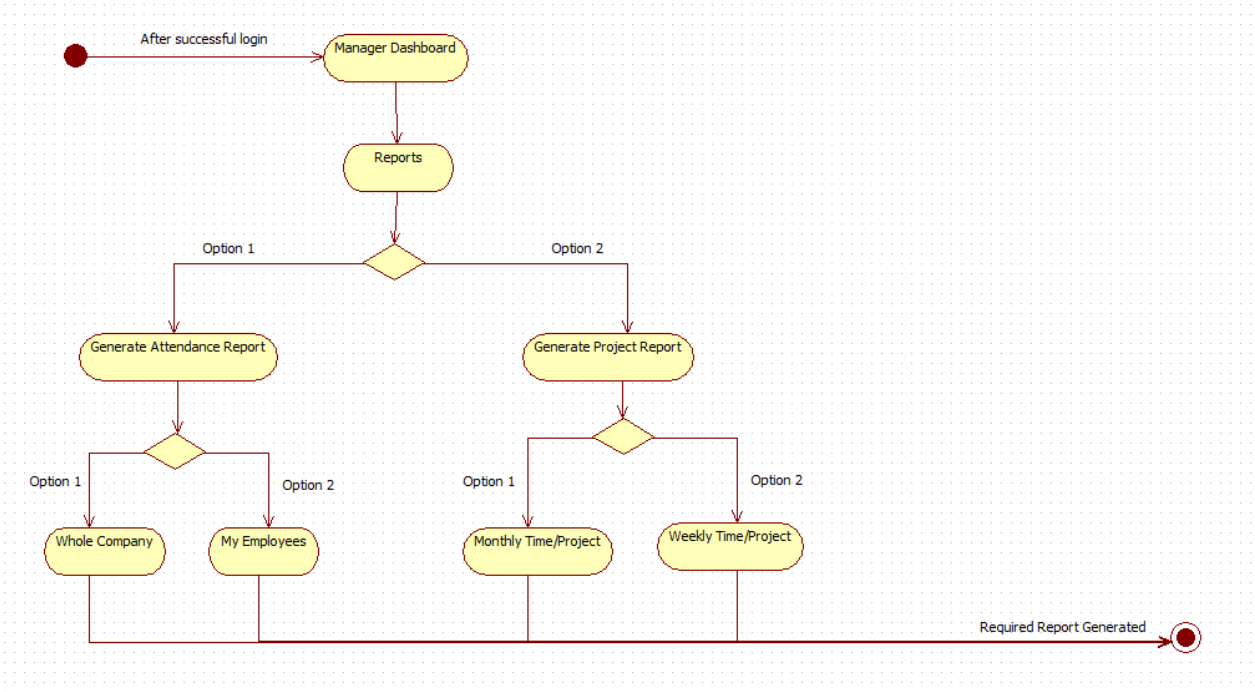
Appendices

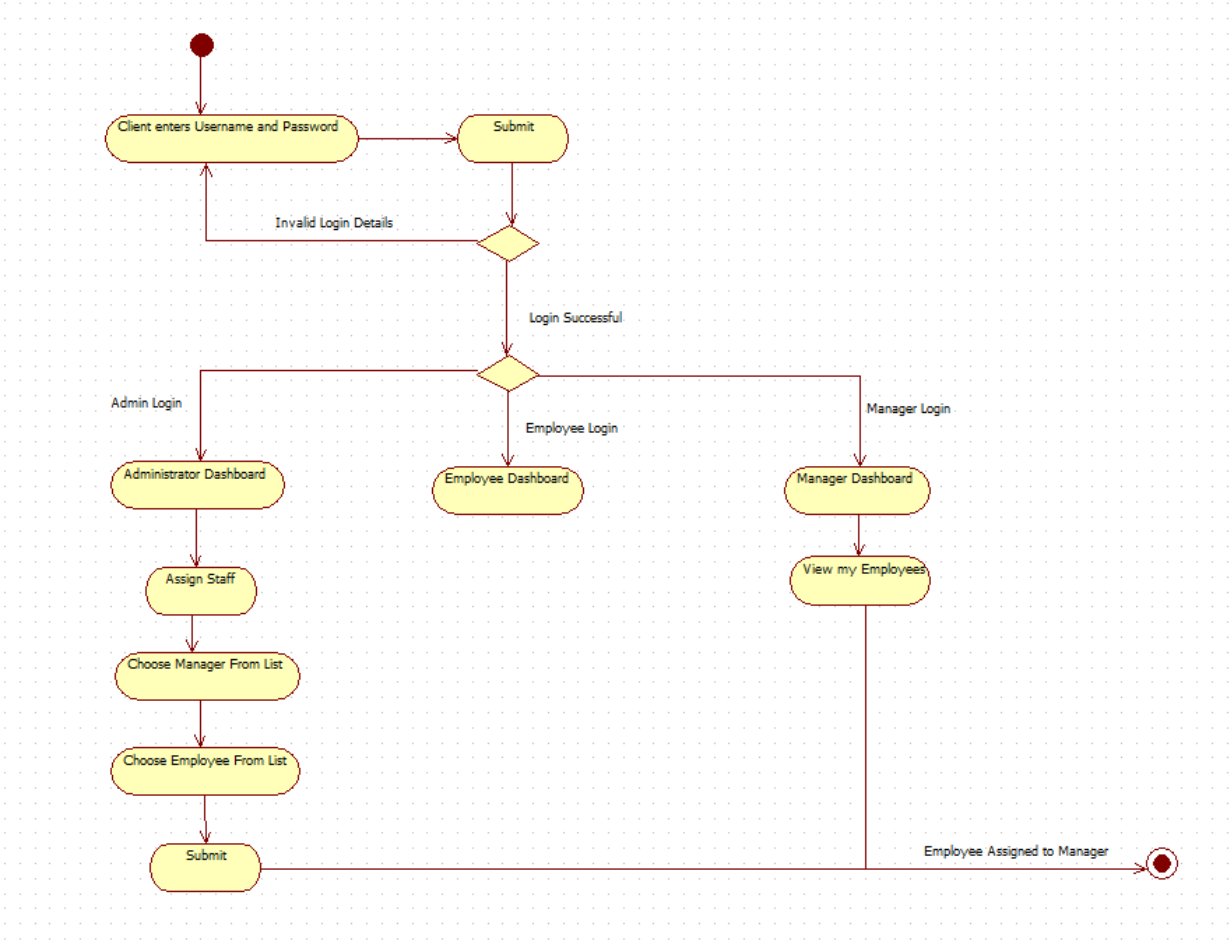
Wireframes

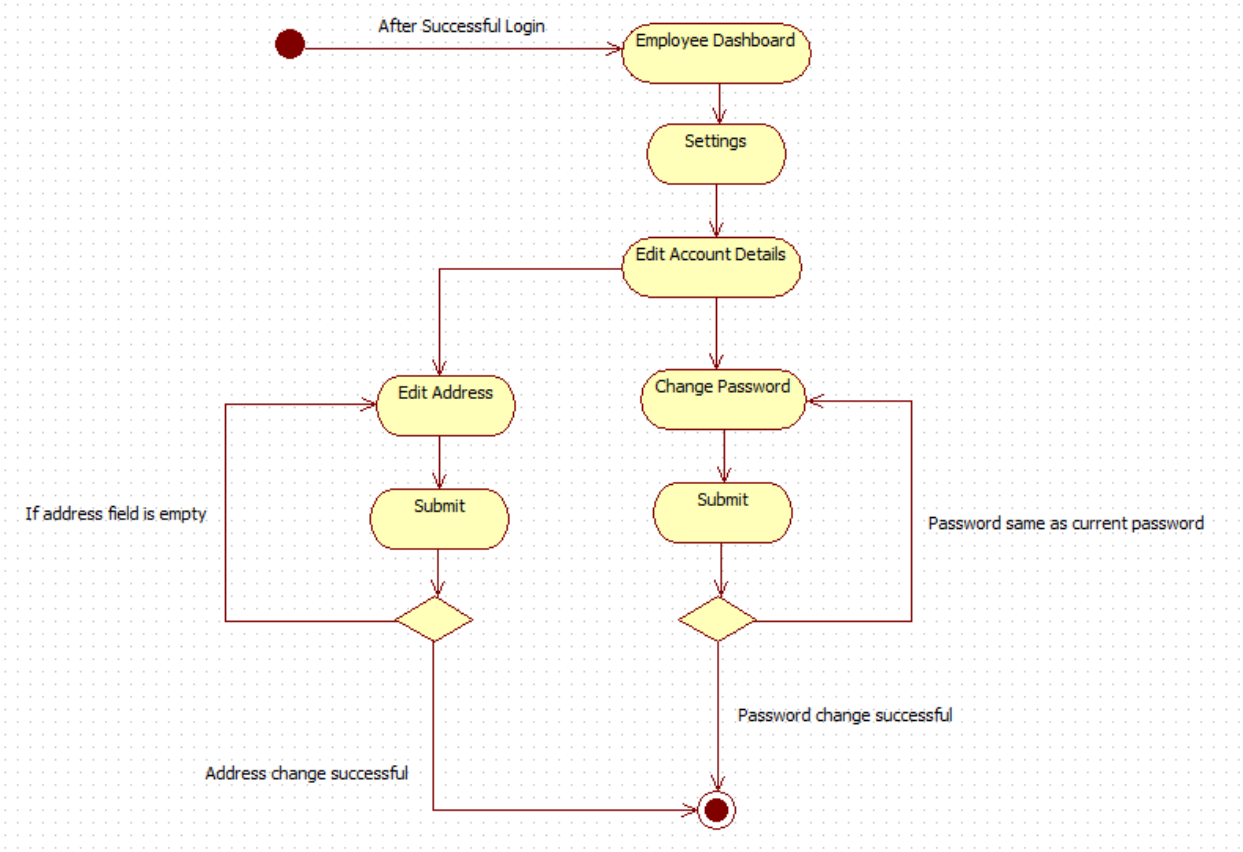




Activity Diagrams






After Successful Login
Employee Dashboard
Settings
Edit Account Details
Edit Address
Change Password
Submit
Submit
If address field is empty
Password same as current password
Password change successful
Address change successful

Use-Case Scenarios

ID	4	
Name	Add Project	
Summary	The following steps are required for the Manager to successfully add a project	
Priority	Very High	
Preconditions	The completion of Use case scenario id 1	
Post conditions	The Project has been added and assigned to the manager	
Primary Actor(s)	Manager	
Secondary Actor(s)	N/A	
Trigger	Manager can add Project to the system	
Main Scenario	1	The interface displays a manager related section with the button for 'add new project'.
	2	The manager clicks on the button
	3	The system will present the Manager with a form to fill out
	4	The Manager clicks submit
	5	The system adds the project to the project table and assigned project table with relevant manager.
	6	The system notifies the manager of the successful adding of the project
Extensions	6a – If any of the input fields are invalid, the system will display an error message detailing the invalid fields. 6b – If the Project start time is before today's date, the system will throw an error message and not insert the record.	

ID	5	
Name	Add Deliverable	
Summary	The following steps are required for the Manager to successfully add a Deliverable	
Priority	Very High	
Preconditions	The completion of Use case scenario id 1	
Post conditions	The deliverable has been added and assigned to the project	
Primary Actor(s)	Manager	
Secondary Actor(s)	N/A	
Trigger	Manager can add Deliverable to the Project.	
Main Scenario	**1**	The interface displays a manager related section with the button for 'add new deliverable.
	2	The manager clicks on the button
	3	The system will present the Manager with a form to fill out
	4	The Manager clicks submit
	5	The system adds the deliverable to the deliverable table with relevant project.
	6	The system notifies the manager of the successful adding of the deliverable
Extensions	6a – If any of the input fields are invalid, the system will display an error message detailing the invalid fields. 6b – If the Deliverable start time is before today's date, the system will throw an error message and not insert the record.	

ID	6	
Name	Add Task	
Summary	The following steps are required for the Manager to successfully add a Task	
Priority	Very High	
Preconditions	The completion of Use case scenario id 1	
Post conditions	The task has been added and assigned to the deliverable	
Primary Actor(s)	Manager	
Secondary Actor(s)	N/A	
Trigger	Manager can add task to the deliverable.	
Main Scenario	**1**	The interface displays a manager related section with the button for 'add new task'.
	2	The manager clicks on the button
	3	The system will present the Manager with a form to fill out
	4	The Manager clicks submit
	5	The system adds the task to the task table with relevant deliverable.
	6	The system notifies the manager of the successful adding of the deliverable
Extensions	6a – If any of the input fields are invalid, the system will display an error message detailing the invalid fields. 6b – If the task start time is before today's date, the system will throw an error message and not insert the record.	

Database Tables

Users

100	Guinevere	Walker	P.O. Box 557, 7606 Mauris Rd.	Onze-Lieve-Vrouw-Lombeek	7841	0	2760	16-06-17	gwalker	NULL	0
102	Tyler	Watts	31 Easby Crescent	Morden	SM46SB	07792320402	JP453215P	05-06-89	twatts	ccd5faaf2184d4787a0fb0caac68d1546375d0f6bfa9b749b2...	0
105	Graeme	Jones	224 Kingston Avenue	Kingston, London	KT9 5SP	0	JP1234567	06-01-1975	gjones	ccd5faaf2184d4787a0fb0caac68d1546375d0f6bfa9b749b2...	1
107	Tina	Watson	5 Lordswood House	18 Cayton Road, Coulsdon, Surrey	CR51LT	0	JP453215P	1969-05-06	twatson	ccd5faaf2184d4787a0fb0caac68d1546375d0f6bfa9b749b2...	0
108	Jamie	watts	123 Street	Morden	SM46SN	0	JP156273	1993-02-04	jwatts	ccd5faaf2184d4787a0fb0caac68d1546375d0f6bfa9b749b2...	0
109	Brad	Jones	123 Bradacre Road	Morden	SM4 5RK	0	JP118832	2016-12-23	bjones	ccd5faaf2184d4787a0fb0caac68d1546375d0f6bfa9b749b2...	0
110	Olivia	Watson	18 Cayton Road, 4 Lordswood House	Coulsdon	CR51LT	0	JP781233L	01/12/1992	owatson	ccd5faaf2184d4787a0fb0caac68d1546375d0f6bfa9b749b2...	2
111	Arturas	Bulavako	78 Station Avenue	Ewell	KT9 7UG	0	JP116611D	2017-03-02	abulavako	ccd5faaf2184d4787a0fb0caac68d1546375d0f6bfa9b749b2...	0
112	Graeme	Watts	31 Easby Crescent	Morden	kt17 2he	0	JP90892S	2017-03-02	gwatts	ccd5faaf2184d4787a0fb0caac68d1546375d0f6bfa9b749b2...	0
113	Bradley	De Stockton Snoddy	34 chadacre road	stoneleigh	kt17 2he	0	88888888	2017-03-03	bsnoddy	cc27219c548fd72b25f6e33bb1efd169799eab76ac10d4d411...	0
114	Olivia	Watson	4 Lordswood house 18 Cayton Road	Coulsdon	CR51LT	0	JP453215P	1993-04-24	owatson1	ccd5faaf2184d4787a0fb0caac68d1546375d0f6bfa9b749b2...	0

Clockings

	Clock_ID	User_ID	In_Out	Reference	dts
Edit Copy Delete	1	118	1	NULL	2017-04-13 18:37:53
Edit Copy Delete	2	102	1	NULL	2017-04-13 18:39:55
Edit Copy Delete	3	102	1	NULL	2017-04-14 00:01:58
Edit Copy Delete	4	120	1	NULL	2017-04-14 00:02:41
Edit Copy Delete	5	113	1	NULL	2017-04-14 00:05:44
Edit Copy Delete	6	107	1	NULL	2017-04-14 18:21:14
Edit Copy Delete	7	102	1	NULL	2017-04-18 13:59:44
Edit Copy Delete	8	102	1	NULL	2017-04-19 17:34:37
Edit Copy Delete	9	102	1	NULL	2017-04-20 01:06:13
Edit Copy Delete	10	102	0	NULL	2017-04-20 01:12:33
Edit Copy Delete	11	102	1	NULL	2017-04-20 01:25:47
Edit Copy Delete	12	120	1	NULL	2017-04-21 01:33:28

Deliverables

	Deliverable_ID	Project_ID	Name	Description	Deadline
Edit Copy Delete	1	10	Home Page	Complete fully responsive site to show client in m...	2017-01-31
Edit Copy Delete	2	10	Finish About Page	Fully Complete the About page for the web applicat...	2017-05-10
Edit Copy Delete	4	12	Search Functionality	Add search functionality to the web application	2017-05-24
Edit Copy Delete	5	116	Filters on Archive	Finish the responsiveness on the filters on archiv...	2017-04-26

AssignedProject

	Project_ID	Manager_ID	Date_Assigned
Edit Copy Delete	10	18	2017-01-04
Edit Copy Delete	12	18	2017-03-24
Edit Copy Delete	30	18	2016-12-25
Edit Copy Delete	40	18	2017-04-12
Edit Copy Delete	116	18	0000-00-00
Edit Copy Delete	117	1	0000-00-00
Edit Copy Delete	118	1	0000-00-00
Edit Copy Delete	119	1	0000-00-00
Edit Copy Delete	120	18	0000-00-00

AssignedEmployees

	Employee_ID	Manager_ID
Edit Copy Delete	6	8
Edit Copy Delete	10	18
Edit Copy Delete	87	18
Edit Copy Delete	88	1
Edit Copy Delete	91	18
Edit Copy Delete	95	18
Edit Copy Delete	96	18
Edit Copy Delete	98	18
Edit Copy Delete	99	18
Edit Copy Delete	100	18
Edit Copy Delete	101	18

Employees

	Employee_ID	Job_Title	User_ID
Edit Copy Delete	1		1
Edit Copy Delete	2		2
Edit Copy Delete	3		3
Edit Copy Delete	4		4
Edit Copy Delete	5		5
Edit Copy Delete	6		6
Edit Copy Delete	7		7
Edit Copy Delete	8		8
Edit Copy Delete	9		10
Edit Copy Delete	10		11
Edit Copy Delete	11		12
Edit Copy Delete	12		13
Edit Copy Delete	13		14
Edit Copy Delete	14		15
Edit Copy Delete	15		16
Edit Copy Delete	16		17
Edit Copy Delete	17		18
Edit Copy Delete	18		20
Edit Copy Delete	19		21
Edit Copy Delete	20		23

Holiday

	Holiday_ID	Reference	Days	Hours
Edit Copy Delete	1	This is a test	*NULL*	*NULL*
Edit Copy Delete	2	This is a test	*NULL*	*NULL*
Edit Copy Delete	3	fjdghfjdghfjdg	*NULL*	*NULL*
Edit Copy Delete	4	rkdjtkrjtkrjdtjrktjre	*NULL*	*NULL*
Edit Copy Delete	5		*NULL*	*NULL*

HolidayRequest

	Holiday_ID	Employee_ID	Manager_ID	Status	Date_From	Date_To
Edit Copy Delete	1	87	18	0	2017-04-22	2017-04-30

Manager

+ Options

	Manager_ID	User_ID	dept_id
Edit Copy Delete	1	4	1
Edit Copy Delete	2	9	2
Edit Copy Delete	3	19	NULL
Edit Copy Delete	4	22	NULL
Edit Copy Delete	5	30	NULL
Edit Copy Delete	6	32	NULL
Edit Copy Delete	7	36	NULL
Edit Copy Delete	8	42	NULL
Edit Copy Delete	9	48	NULL
Edit Copy Delete	10	54	NULL
Edit Copy Delete	11	59	NULL
Edit Copy Delete	12	66	NULL
Edit Copy Delete	13	69	NULL
Edit Copy Delete	14	79	NULL
Edit Copy Delete	15	86	NULL
Edit Copy Delete	16	92	NULL
Edit Copy Delete	17	96	4
Edit Copy Delete	18	105	3

Overtime Request

#	Name	Type	Collation	Attributes	Null	Default	Extra	Action
1	Overtime_ID	int(11)			No	None	AUTO_INCREMENT	Change Drop Primary Unique Index Spatial Fulltext Distinct values
2	Days	int(11)			No	None		Change Drop Primary Unique Index Spatial Fulltext Distinct values
3	Hours	int(11)			Yes	NULL		Change Drop Primary Unique Index Spatial Fulltext Distinct values

Overtime

#	Name	Type	Collation	Attributes	Null	Default	Extra	Action
1	Overtime_ID	int(10)			No	None		Change Drop Primary Unique Index Spatial Fulltext Distinct values
2	Manager_ID	int(10)			No	None		Change Drop Primary Unique Index Spatial Fulltext Distinct values
3	Employee_ID	int(10)			No	None		Change Drop Primary Unique Index Spatial Fulltext Distinct values
4	Status	int(1)			No	None		Change Drop Primary Unique Index Spatial Fulltext Distinct values

Project

	Project_ID	Name	Start_Date	Deadline
Edit Copy Delete	1	Project 1	2016-12-01	2017-01-01
Edit Copy Delete	2	Project 2	2016-12-01	2017-01-03
Edit Copy Delete	3	Project 3	2016-12-15	2017-01-01
Edit Copy Delete	5	Project 4	2016-12-08	2017-01-08
Edit Copy Delete	7	Project 5	2016-12-11	2017-02-07
Edit Copy Delete	8	Project 6	2016-12-09	2016-12-10
Edit Copy Delete	9	Project 7	2016-12-09	2016-12-24
Edit Copy Delete	10	Project 8	2016-12-14	2017-02-08
Edit Copy Delete	11	Project 9	2016-12-21	2017-01-18
Edit Copy Delete	12	Project 10	2017-01-11	2017-02-22

Shift

	ShiftID	User_ID	Start	End
Edit Copy Delete	1	102	09:00:00	17:00:00
Edit Copy Delete	2	108	11:00:00	19:00:00

Task

Task_ID	Deliverable_ID	Name	Description	Date_Created	start_date	end_date
1	2	PHP Functionality	Please ensure the functionality is fully operation...	2017-03-24	NULL	NULL
3	2	Search bar functionality	The search bar will need to be ajax on this page	2017-04-19	NULL	NULL

Time_Logs

#	Name	Type	Collation	Attributes	Null	Default	Extra
1	Log_ID	int(11)			No	*None*	AUTO_INCREMENT
2	User_ID	int(11)			No	*None*	
3	Task_ID	int(11)			No	*None*	
4	Check_In	datetime			No	*None*	
5	Check_Out	datetime			No	*None*	

Project Proposal

Abstract

This project is being undertaken in an attempt to produce a high quality, feature rich, dynamic employee management system for use by multiple expanding companies. The expansion of a company demands a procedure in which managers can effectively communicate with employees, whilst obtaining relevant information regarding the progress of projects. Although multiple systems are available to achieve one or more of these outcomes, there is yet to be a cost-effective system in place for up-and-coming companies to fully take advantage of. Fortunately, an agreement has been reached between an IT company and myself for the development of such a product.

Introduction and Background

Overview

The main task at hand during this project is to provide a Web solutions company with a web application that can better aid them in their management of employees. They require a way of better handling requests from employees and also improve on their current system of monitoring time against product.

Currently, they have a web application in which an employee signs in and manually writes their activities and the length of time. The improved system will allow managers to set projects and employees to pick from one, automatically tracking the time against each project.

Background

My Client is a Web Solutions and Telecoms Provider company named Staxoweb. The company as a whole consists of multiple departments (Development, Infrastructure, Finance, Digital Marketing and Sales), each of which has a manager. They wish to possess a web application that can better help them analyse the amount of time each project is taking to reach completion. Furthermore, their requirements state the need to be able to have an automated system that can process holiday, overtime, and sick pay requests from employees.

Currently, Staxoweb uses a bespoke web application to process some of the above requirements, but desire a more secure 'centralised hub' in which all features are easily accessible; with the addition of analysis time tracking qualities. Implementing an employee management system will alleviate the problems the company is currently experiencing when trying to determine the detrimental aspects of their development process.

Stakeholders

The predominant beneficiaries of this project will be the managers of each department (Internal stakeholders). The fundamental point taken from the project brief was the need for managers to monitor project progression. The addition of this functionality will benefit managers as they can action productivity enhancing practises to key areas where yield is halting. Additionally, their employees will benefit greatly from having a centralised area where they can view upcoming deadlines and request holiday, overtime, and sick pay without having to manually arrange a meeting with their managers.

In addition to internal stakeholders profiting from this system, external stakeholders such as their clients may also experience an overall improved customer service. Improved project completion time may result in satisfied customers and advance business productivity across multiple internal sectors.

This software as a whole could benefit many other users other than the employees at Staxoweb. The need for a robust communication/management tool that can cater for many additional features (potentially many more after version 1), is in high demand across businesses of many sectors. Employees of a supervisory status could profit from such a utensil, and the potential outcome of increased productivity is extremely appealing to stakeholders within a business.

Aims and Objectives

The predominant goal of this project is to enable a company to efficiently manage their employees by means of analysis aid and automated communication requests.

The main aim for this project is to help improve a Company with a current temperamental system and provide them with a system that handles employee requests and project management functionality, increasing the general productivity of the business.

The addition of the bulletin board regarding projects will increase awareness of the company's current projects/deadlines across both managers and employees and will aid in consistency across multiple departments.

Objectives

Managers must be able to add a list of projects with a project description and any other relevant information to aid the employee with understanding exactly what needs to be achieved. Employees must also be able to view the projects in a select list on their main user interface and 'check in' with the corresponding project, therefore logging a record of time against said project. The completion of this objective is the main goal in producing the project analysis tools the company requests.

Managers must have a project analysis section on their interface which shows current number of hours spent on said project against time permitted for project completion. This should be displayed as figures, and have the option to be displayed in a graphical format. This should reduce product completion time in the first 3 months by at least 5%.

Managers must be able to set a start and end date for each project and these end times must list on the corresponding employee's bulletin board in order of nearest deadline. This can be achieved with an SQL query to the database with the correct parameters and formatting. This list must be updated within 10 seconds of the manager submitting a new project date range.

Employees need to have the option to send requests to their respective manager in the form of holiday, overtime, and sick pay applications. The managers need to then receive a notification of said request and have the option to accept or decline, with the added option of a note for clarification purposes. Once the outcome has been decided, the employee would be notified and has a log of the request in their corresponding section of the interface. This would resolve any overlap of employee granted requests and decrease the response times for said requests by 25%.

A desire of my client would also be to add the seamlessness of communication between manager and employee without the need for a third party communication tool such as Outlook. A possible way of achieving this would be the implementation of a messaging API. This is not a requirement at this time, but may be a feature for a later iteration.

Technologies & Resources

Applications

Atom

Atom will be the primary text editing program used to input the programming code. Atom boasts a large community in which packages are created and can be downloaded and installed. These packages include a 'Remote FTP' which allows files to be directly uploaded to the server whilst also duplicating the files locally in case of a network error whilst uploading. The user interface is clean, responsive and easy to navigate.

MySQL

MySQL is a widely active open source database that is used to securely create, read, update, and delete vital information required for applications. Custom queries can be implemented to achieve this and transmit relevant information in the application. ***(Top reasons to use MySQL, 2016)***

StarUML

This software will be used to build the design of the database. It will add a visual aid to display the flow of data and interaction between database elements to complete the desired outcomes. This software is open source and is academically approved.

Lucid Chart

Lucid Chart is an online resource for constructing flow charts and further design based diagrams. This resource will be used throughout this project as a tool to build significantly accurate wireframes for each user type interface.

Scripting Languages

PHP

PHP is a Server-side HTML embedded scripting language that can be used to generate dynamically used content on webpages. PHP is not limited to generating only HTML, but can also output pdf, images and other content.

HTML

Hypertext Mark-up Language (HTML) is essentially a formatting language consisting of tags, hyperlinks and other elements. HTML is rendered by the browser and interpreted into colours, fonts, links and further components that make up the structure of a webpage.

BOOTSTRAP

Bootstrap is a fully responsive HTML, JavaScript and CSS framework. It is extremely popular for developing responsive webpages for use on multiple devices. The column based class system allows for dynamic and consistent design across numerous window widths, and the JavaScript library allows for the implementation of convenient functionality on components.

CSS

Cascading style sheets are the standardized way of implementing styling to webpages. Including a link tag in the head of the html document linking to a CSS sheet, allows for all styling code to be included in one place and interpreted by the browser. Different browsers have diverse ways of

processing CSS, which is why browser testing is so important when it comes to 'front-end' snagging. My project will require

JavaScript/JQuery

JavaScript is a programming language that is executed at the browser level, and is used to add interactivity and dynamism into webpages. The jQuery library is an extension of JavaScript and can be used to transition elements and add functionality to the overall website. These are considered as essential components in the building of a successful web application, and therefore will be used frequently throughout my project.

Resources

Data

My client will be providing the details of all employees currently under contract. The company consists of approximately 15-20 employees, of which are 5 managers/supervisors etc. This would be the minimal amount of data expected within a system such as this. Due to this software being built to potentially appeal to other businesses that may contain more employees, it is vital that my database is strenuously tested with more records than are perhaps necessary. For this I will be using online resources stated below.

Important variables to consider when inputting data are aspects such as query speeds, reliability and accuracy of data displayed, and overall database performance. To ensure these requirements are met, a slight abundance of the recommended amount of data shall be included so to fully optimize any issues that arise.

Generate Data

Stress Testing is a core component during the Testing phase. A sizeable amount of 'dummy' data may be essential for replicating the larger number of users the database will experience when the project is completed. To apply this, a third party website such as ' http://www.generatedata.com/ ' will be used to import useful (correctly formatted) data. ***(Generatedata.com, 2006)***

Hammer DB

A secure database is essential when personal information is involved. Hammer DB is a graphical open source load testing and benchmarking software for databases to ensure performance is meeting the high standards required for a secure and fully functional database - ' http://www.hammerdb.com'. This will be used to test and address any issues that may compromise the safety of the database. ***(HammerDB, no date)***

Browser Stack

Front end testing a website is extremely important before releasing the final product. Browsers process specific elements (CSS mostly) in slightly different ways. To recognize this and fix the issues, a third party resource such as BrowserStack is required. It runs the URL through 100% manufacturer approved simulators to replicate the display of the webpage across multiple devices. A tool like this is essential in noticing these matters and rectifying them, to prevent any inconsistencies in the final iterations. – 'http://www.browserstack.com'. ***(j Query, 2011)***

1. Methodology

1.1. Project Methodology

The importance of an appropriate project methodology is a crucial factor to having effective project management and as a result, reflects positively in the quality of the finished product. Sequential methodologies such as Waterfall, Critical Path Method (CPM) and Critical Chain Project Management (CCPM), all have the core values of following an unwavering procedure from beginning to end. Although beneficial to some extent, the risks are far greater and does not allow for much modification during the development process. The values held by the agile methodologies are far more suited to my project.

The methodology used for my project will be the agile methodology using scrum framework and adhering to the principles within Scrum. Scrum essentially splits components into separate periods of time called Sprints. Within Sprints are a cycle of development processes; Requirements Analysis, Project Design, Implementation, and Testing. The theory is that by the end of each Sprint, the resulting product will have undergone such attention to detail that it'll be less likely to fail. This iteration process of sprinting is repeated throughout the entire project and allows for easy modifications to be made if required. The analysis sections within the sprints result in an increased time with the Client. This also means that as a developer, you gain a greater insight into the wants and needs of said client, and the finished product is related far more closely to their vision. ***(Babu Yegi, 2016)***

The client focused aspect of this methodology appeals specifically to my project. Having previously been employed by my client, my knowledge of their reliability and the abundance of communication methods between them and me make for easy accessibility when gathering new requirements and presenting updated prototypes (components). This allows for consistency and the ability to follow the agile methodology throughout development.

Risks & Contingences

Identifying risks within a project, and preparing solutions is a key factor in the project life-cycle. Regarding my project, there are a potential number of risks that could hinder progress. This is partially due to the reliance on third party services. My project is hosted with a company based in the UK called ***Smart Hosting***. They boast a 99.9% reliability rating, and have a mass of positive reviews; however, this is not always a certainty for quality ***(Web, VPS and reseller hosting, no date).*** A possible contingency plan to combat the downtime of a server is the possibility of downloading local copies of the database and arranging a synchronization process every 6/12 hours. This results in the client still being able to access important data, and ensures the system is still operational.

A more general risk involved when producing work for a client, is always the possibility of the Client changing their mind and cancelling the project all together. This is an unpreventable event should the client no longer desire the product, however, frequent conversing and reassuring the client would help reduce the chances of such a situation occurring. This is also an advantage to following the agile methodology principles, which focusses heavily on the needs and wants of the client.

Ethics & Data Protection

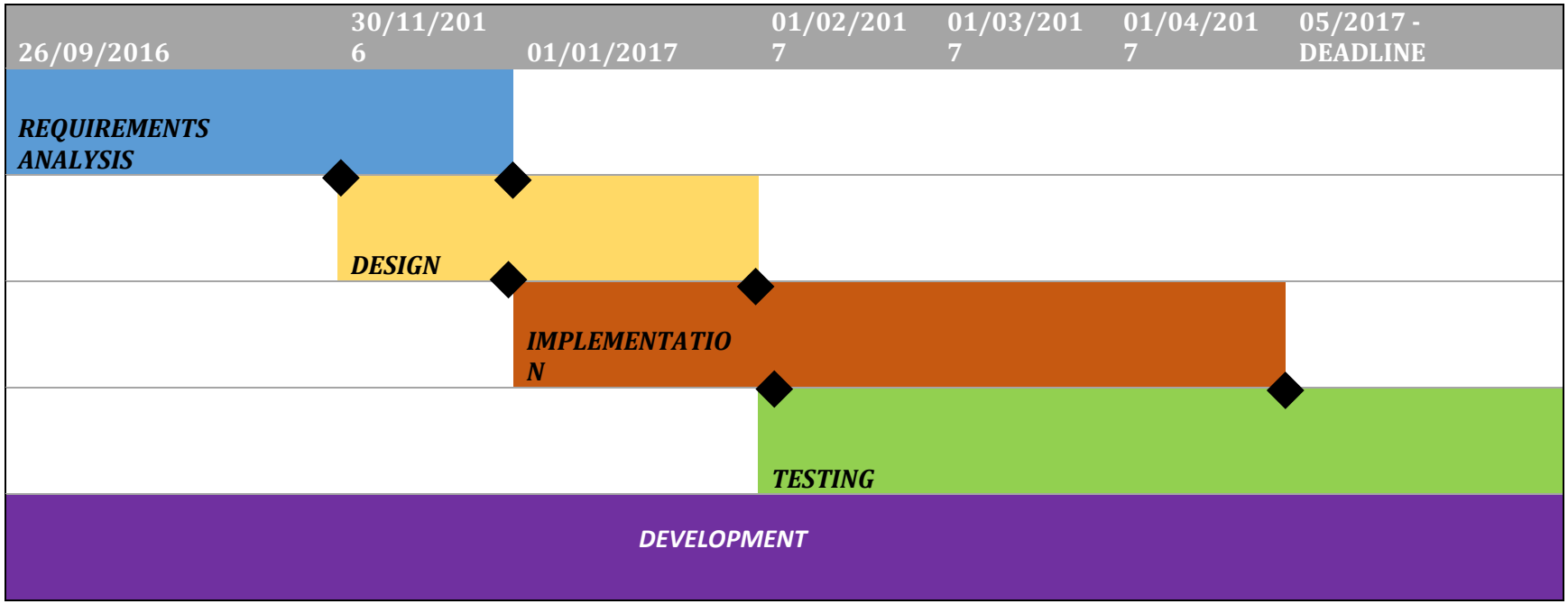
A business that contains a database populated with user's personal details has to adhere to guidelines as to not breach any data protection laws and remain ethical. As stated in the Data Protection Act, a business is responsible for the use of all data, and has to follow a set of rules called 'data protection principles'. My project will be storing sensitive data such as employee's addresses; national insurance numbers, and contact details and will, of course be following these principles. ***(Data protection, 2016)***

Data that has expired has to be deleted in the correct manner and kept no longer than necessary. This will have to be made aware to my clients as this could directly impact them if an employee leaves the company. MySQL also state that they '***offer exceptional security features that enable absolute data protection'***. This consists of SSH and SSL certificates for the added security of sensitive data flow.

Project Management

Gantt Chart

Any successful project using the agile scrum methodology requires an accurate, phase-based system in which deliverables are categorized. My project will be split into four separate phases, which will each be activated as substantial milestones are met.



Above is a GANNT chart displaying the main phases and the times in which they will be implemented throughout the development process. The black diamond-shaped symbols represent these significant milestones reached, and therefore triggering the prompt for the next phase. Within these phases, similarly built cycles will be executed to ensure components are constructed appropriately and are adherent to the agile practices.

As shown above, many of the phases overlap slightly. This is to ensure a consistent progression is met, and that the highest level of quality is applied with all the functionalities included.

Milestones

Below are the significant milestones displayed in the GANNT chart. These milestones are located both midway, and the end of each phase. These substantial milestones result in the start of the next phase.

Milestone	Date	Phase
Initial Requirements Analysis	30/11/2016	Requirements Analysis
Final Requirements gathered from design iterations	01/01/2016	Requirements Analysis
Initial Designs (Wireframes, flow chart)	01/01/2016	Design
Finalized Designs	01/02/2016	Design
User Interface for manager and employee with some functionality (1st Iteration)	01/02/2016	Implementation
Continuous Testing of implementation iterations from this point forward	01/02/2016	Testing
Completed project, implemented	01/05/2016	Implementation

Deliverable List

Having a well-structured, thought out deliverable list is essential to ensuring deadlines are met and progress stays consistent. Please see section 6.2 Appendices for detailed deliverable list with highlighted submissions.

Date	Type	Deliverable
30/09/2016	Documentation	Initial Draft Proposal
07/10/2016	Methodology	Initial ERD
07/10/2016	Documentation	Complete draft proposal
14/10/2016	Documentation	SWOT
14/10/2016	**Submission**	**Project Proposal**
21/10/2016	Technology	Install DB software
28/10/2016	Methodology	Stakeholder analysis
28/10/2016	Documentation	TOC Requirements Chapter
04/11/2016	Technology	Establish PHP connection to database
04/11/2016	Technology	Data and Database Creation
11/11/2016	Methodology	Use Cases and User Stories
11/11/2016	Technology	Simple SQL and PHP report

18/11/2016	Methodology	List of Requirements / MoSCoW
25/11/2016	Documentation	Draft Requirements Chapter
25/11/2016	Technology	Bootstrap/Mobile API Interface
02/12/2016	Documentation	TOC Design Chapter
02/12/2016	Methodology	Sketches/Wireframes
09/12/2016	**Submission**	**Early Prototype demonstration**
16/12/2016	Documentation	Requirements Chapter
16/12/2016	Methodology	Navigation/Sequence diagrams
23/12/2016	Methodology	Identify SQL queries from requirements
23/12/2016	Technology	Implement mock-ups of key webpages
06/01/2017	Methodology	Revised ERD
06/01/2017	Methodology	Data Dictionary
06/01/2017	Technology	Key report 1
13/01/2017	Documentation	Draft Design Chapter
13/01/2017	Technology	Key form 1
20/01/2017	Methodology	Test Strategy
20/01/2017	Documentation	Identify Review Topic
27/01/2017	Technology	Key report 2
27/01/2017	Documentation	Design Chapter
03/02/2017	Technology	Key form 2
03/02/2017	Documentation	TOC Review Chapter
10/02/2017	Technology	Identify most complex SQL
17/02/2017	Documentation	Draft Review Chapter
24/02/2017	Technology	Key report 3
03/03/2017	Technology	Key form 3
03/03/2017	Documentation	TOC Implementation Chapter
03/03/2017	Documentation	Review Chapter
10/03/2017	Technology	Key report 4
10/03/2017	Documentation	Technologies and Architecture Section
17/03/2017	Technology	Key form 4
17/03/2017	Documentation	Second Major Section
24/03/2017	Documentation	Draft Implementation Chapter
24/03/2017	Technology	Key report 5
31/03/2017	Documentation	Testing Chapter
31/03/2017	Technology	Key form 5
07/04/2017	Technology	Identify viva-oriented functionality
07/04/2017	Documentation	Implementation Chapter

11/04/2017	Documentation	Introduction and Conclusion
14/04/2017	Documentation	Appendices and references
21/04/2017	Technology	Last functionality implemented
21/04/2017	Documentation	Complete draft report
24/04/2017	**Submission**	**Report Submission**

6.5 Methodology Comparison

Please see Arguments between Waterfall/AGILE methodologies below.

	Waterfall	***AGILE***
Advantages	- Requirements and stages are fixed and concise. - No overlapping of phases - Simple to understand and resource is marginal compared to various other project management methodologies.	- Stakeholders can visually see functioning software and have the ability to make adjustments if desired. - Predictability throughout various aspects of project building (schedule, quality, deadlines) - Developer gets to understand the business and therefore a more detailed account of the client's wants.
Disadvantages	- Not easy to implement changes once in latter stages of development - High risk due to requirements not being flexible - Software is not implemented until concluding stages of the model	- Potentially more time consuming as frequent meetings have to be scheduled with clients (for iterations and requirement amendments) - Quality may be compromised due to the dependable modifications and speed of the development process